

MySQL

Complete Practical Notes

A hands-on study companion — every topic with runnable
CREATE / INSERT scripts and practice questions to solve.

How to Use These Notes

These notes turn the SQL portion of the Data Analytics syllabus into a complete, self-contained practical course. Every concept is explained in plain language, shown with a **runnable MySQL example**, and followed by **practice questions** you can solve immediately.

Golden rule of this book

✓ **Key Point:** Every worked example and every practice set is **self-contained**. The exact `CREATE TABLE` and `INSERT` statements you need are printed right before the questions, so you can paste them into MySQL Workbench (or the `mysql` client) and start querying — no external database required.

Setting up your practice environment

1. Install **MySQL Community Server 8.0+** and **MySQL Workbench** (or use `mysql` on the command line).
2. Open a connection and create a scratch database: `CREATE DATABASE practice; USE practice;`
3. For each chapter, run the **Setup Script** block first, then attempt the queries.
4. When you finish a chapter, run `DROP DATABASE practice; CREATE DATABASE practice; USE practice;` to start fresh, or keep tables with different names.

Icons used in this book:

- ✓ **Key Point** — a rule worth memorising.
- ✎ **Practice Questions** — solve these before moving on.
- **Setup Script** — the CREATE/INSERT code that powers the examples and questions in that section.

Table of Contents

Part I — Foundations

- 1 Introduction to Databases, DBMS & MySQL
- 2 Database Design & Architecture
- 3 Data Models, ER Diagrams & Normalization

Part II — Defining & Populating Data

- 4 MySQL Data Types & Basic Syntax
- 5 DDL — Creating and Altering Tables
- 6 Constraints, Primary & Foreign Keys
- 7 DML — INSERT, UPDATE, DELETE

Part III — Querying Data

- 8 SELECT and the Core Clauses
- 9 Operators in MySQL
- 10 Dealing with NULL Values
- 11 Sorting, DISTINCT & LIMIT

Part IV — Summarising Data

- 12 Aggregate Functions & GROUP BY
- 13 The HAVING Clause
- 14 Built-in Functions (String, Numeric, Date)
- 15 The CASE Operator & Control Flow

Part V — Combining Data

- 16 Joins in MySQL
- 17 Set Operations: UNION, INTERSECT, EXCEPT

Part VI — Advanced Techniques

- 18 Subqueries
- 19 Common Table Expressions (CTEs)
- 20 Window Functions
- 21 Views
- 22 Stored Procedures & Functions
- 23 Indexes & Query Optimization

24 AI-Assisted SQL (ChatGPT, Gemini, SQLCoder)

Part VII — Practice Papers

10 Practice Papers × 100 Marks (with full setup scripts)

Appendices

Appendix A Solutions to all 10 Practice Papers

Appendix B MySQL Quick Reference Cheat Sheet

Appendix C Answers to Chapter Practice Questions

PART I

Foundations

1. Introduction to Databases, DBMS & MySQL

Before writing a single query, it helps to understand **what a database is** and **why we use one**. Almost every application you use — a shopping site, a bank app, a college portal — stores its information in a database and retrieves it using SQL.

1.1 What is Data and a Database?

Data is raw facts: a name, a price, a date. On its own it means little. **Information** is data organised so it becomes useful — for example, `"total sales in June were ₹2,40,000"`.

A **database** is an organised collection of related data stored electronically so it can be easily accessed, managed and updated. In a **relational database** the data is stored in **tables** (rows and columns), and tables can be related to one another.

✓ **Key Point:** A **table** = rows (records) × columns (fields). Each row is one entity (e.g. one customer); each column is one attribute (e.g. the customer's city).

1.2 DBMS vs RDBMS

A **DBMS** (Database Management System) is software that lets you store and retrieve data. An **RDBMS** (Relational DBMS) is a DBMS based on the relational model, where data lives in related tables and integrity is enforced with keys and constraints. **MySQL, PostgreSQL, Oracle, and SQL Server** are all RDBMSs.

DBMS vs RDBMS at a glance

Feature	DBMS	RDBMS
Data storage	Files / hierarchical	Tables (relations)
Relationships	Not supported well	Keys enforce relationships
Normalization	Not used	Supported
Examples	File systems, XML	MySQL, Oracle, PostgreSQL

1.3 What is SQL?

SQL (Structured Query Language) is the standard language for talking to a relational database. With SQL you can define tables, insert and modify data, and — most importantly for a data analyst — **query** data to answer business questions.

SQL is divided into sub-languages:

- **DDL** (Data Definition Language): **CREATE, ALTER, DROP, TRUNCATE** — define structure.
- **DML** (Data Manipulation Language): **INSERT, UPDATE, DELETE** — change data.
- **DQL** (Data Query Language): **SELECT** — read data.

- **DCL** (Data Control Language): **GRANT**, **REVOKE** — permissions.
- **TCL** (Transaction Control Language): **COMMIT**, **ROLLBACK**, **SAVEPOINT** — transactions.

1.4 Why MySQL?

MySQL is the world's most popular open-source RDBMS. It is fast, free, reliable and used by companies such as Facebook, YouTube and Twitter. In this course we use **MySQL 8.0+**, which supports modern features like **CTEs** and **window functions**.

1.5 Your First Commands

Setup Script — first steps

```
-- Create a database and switch to it
CREATE DATABASE practice;
USE practice;

-- See all databases / tables
SHOW DATABASES;
SHOW TABLES;

-- A tiny table so we have something to look at
CREATE TABLE demo (id INT, message VARCHAR(50));
INSERT INTO demo VALUES (1, 'Hello MySQL');
SELECT * FROM demo;
```

Result:

id	message
1	Hello MySQL

Practice Questions — Chapter 1

- Q1.** In your own words, explain the difference between **data** and **information**.
- Q2.** List the five SQL sub-languages and give one command belonging to each.
- Q3.** Create a database named **college**, switch to it, and confirm it exists with **SHOW DATABASES**;
- Q4.** Create a table **student(roll INT, name VARCHAR(40))**, insert two rows, and select them all.
- Q5.** Which SQL sub-language does **CREATE TABLE** belong to, and which does **SELECT** belong to?

2. Database Design & Architecture

Good queries start with good design. **Database design** is the process of deciding what tables you need, what columns each table has, and how the tables relate. Poor design leads to duplicated data, update anomalies and slow queries.

2.1 The Three Levels of Data Models

Designers usually describe a database at three levels of abstraction:

- **Conceptual model** — the big picture. *What* entities exist (Customer, Order, Product) and how they relate, independent of any technology. Usually drawn as an ER diagram.
- **Logical model** — adds detail: every attribute, its data type category, primary keys, foreign keys and normalization. Still database-independent.
- **Physical model** — the actual MySQL implementation: exact data types (`VARCHAR(50)`, `DECIMAL(10,2)`), indexes, storage engine (InnoDB), etc.

Conceptual vs Logical vs Physical

Level	Answers	Audience	Example artefact
Conceptual	What data?	Business users	High-level ER diagram
Logical	How is it structured?	Data architects	Detailed ERD + normal forms
Physical	How is it stored?	DBAs / developers	CREATE TABLE scripts

2.2 Entities, Attributes and Relationships

An **entity** is a real-world thing we store data about (a Customer). An **attribute** is a property of that entity (customer's email). A **relationship** connects entities (a Customer *places* Orders).

Relationships have a **cardinality**:

- **One-to-One (1:1)** — one row in A matches at most one row in B (Person ↔ Passport).
- **One-to-Many (1:N)** — one row in A matches many in B (Customer → Orders). The most common.
- **Many-to-Many (M:N)** — rows in A match many in B and vice-versa (Students ↔ Courses). Implemented with a **junction/bridge table**.

2.3 A First Look at a Schema

Setup Script — a 1:N relationship

```
-- A minimal, well-designed schema: customers place orders
CREATE TABLE customers (
  customer_id INT PRIMARY KEY AUTO_INCREMENT,
  name        VARCHAR(60) NOT NULL,
  city        VARCHAR(40)
);

CREATE TABLE orders (
  order_id    INT PRIMARY KEY AUTO_INCREMENT,
  customer_id INT NOT NULL,          -- foreign key column
  amount      DECIMAL(10,2),
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
```

```
);
```

✓ **Key Point:** The **foreign key** `orders.customer_id` points at `customers.customer_id`. This enforces that you cannot insert an order for a customer who does not exist — that guarantee is called **referential integrity**.

Practice Questions — Chapter 2

- Q1.** Give a real-world example of a 1:1, a 1:N and an M:N relationship (other than the ones above).
- Q2.** For a library, list the entities you would need and the relationships between them.
- Q3.** Which model level decides that a phone number should be `VARCHAR(15)` — conceptual, logical, or physical?
- Q4.** Design (on paper) two tables `authors` and `books` with a 1:N relationship, then write the CREATE statements.
- Q5.** Explain in one sentence what referential integrity protects you from.

3. Data Models, ER Diagrams & Normalization

This chapter goes deeper into **ER diagrams** and the single most important design skill: **normalization** — organising columns so that each fact is stored exactly once.

3.1 Reading an ER Diagram

In an **Entity-Relationship Diagram**, rectangles are entities, ovals (or listed columns) are attributes, and lines are relationships. **Crow's-foot** notation shows cardinality: a single bar means "one", a three-pronged "crow's foot" means "many".

Crow's-foot cardinality symbols

Symbol	Meaning
(bar)	Exactly one
°	Zero or one
< (crow's foot)	Many
°<	Zero or many

3.2 Keys

- **Primary Key (PK)** — a column (or set) that uniquely identifies each row. Cannot be NULL, cannot repeat. Each table should have exactly one.
- **Candidate Key** — any column that *could* serve as the PK (e.g. both **email** and **phone**).
- **Composite Key** — a primary key made of two or more columns together.
- **Foreign Key (FK)** — a column that references the PK of another (or the same) table.
- **Surrogate Key** — an artificial key (like an **AUTO_INCREMENT id**) with no business meaning.

3.3 Why Normalize? The Anomalies

Imagine storing orders in one big flat table that repeats the customer's name and city on every row. Three problems appear:

- **Insertion anomaly** — you cannot add a customer until they place an order.
- **Update anomaly** — if a customer moves city you must update many rows; miss one and the data is inconsistent.
- **Deletion anomaly** — deleting a customer's only order also erases the customer's details.

Normalization removes these by splitting data into related tables.

3.4 The Normal Forms

First Normal Form (1NF)

Every column holds a **single, atomic value** and each row is unique. No repeating groups or comma-separated lists in a cell.

Result:

Bad (violates 1NF)	phones = '99999, 88888'
Good (1NF)	one phone per row, or a separate phones table

Second Normal Form (2NF)

Be in 1NF **and** every non-key column depends on the **whole** primary key (matters only when the PK is composite). Remove **partial dependencies**.

Third Normal Form (3NF)

Be in 2NF **and** no non-key column depends on another non-key column (remove **transitive dependencies**). e.g. don't store **city** and **city_pincode** together if pincode depends on city.

✓ **Key Point:** A practical mantra: "Each non-key column must depend on the key, the whole key, and nothing but the key." That sentence is 1NF→2NF→3NF in a nutshell.

3.5 Denormalization

Denormalization deliberately adds some redundancy back (e.g. storing a pre-computed **total_amount**) to make **reads faster**. It is a trade-off used in reporting/analytics systems where fast queries matter more than update efficiency.

3.6 Worked Example — Normalizing a Flat Table

Setup Script — the problem table

```
-- UN-normalized (bad): repeats customer & product info
CREATE TABLE sales_flat (
  order_id INT, cust_name VARCHAR(50), cust_city VARCHAR(40),
  product VARCHAR(50), price DECIMAL(8,2), qty INT
);
INSERT INTO sales_flat VALUES
(1, 'Aarav', 'Mumbai', 'Earbuds', 4999, 2),
(2, 'Aarav', 'Mumbai', 'Book', 650, 1),
(3, 'Diya', 'Surat', 'Earbuds', 4999, 1);
```

Normalized into three tables:

The 3NF solution

```
CREATE TABLE customers (
  customer_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50), city VARCHAR(40));

CREATE TABLE products (
  product_id INT PRIMARY KEY AUTO_INCREMENT,
  product VARCHAR(50), price DECIMAL(8,2));

CREATE TABLE orders (
  order_id INT PRIMARY KEY AUTO_INCREMENT,
  customer_id INT, product_id INT, qty INT,
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
  FOREIGN KEY (product_id) REFERENCES products(product_id));
```

Practice Questions — Chapter 3

- Q1.** Define primary key, candidate key and foreign key in your own words.
- Q2.** A table stores `student_id`, `name`, `course1`, `course2`, `course3`. Which normal form does it violate and why? Redesign it.
- Q3.** Give an example of a transitive dependency and show how 3NF removes it.
- Q4.** When would you deliberately **denormalize** a table? Give one analytics scenario.
- Q5.** Take the `sales_flat` table above and write the `INSERT` statements that populate the three normalized tables with the same data.

PART II

Defining & Populating Data

4. MySQL Data Types & Basic Syntax

Choosing the right **data type** for each column keeps data valid and storage efficient. MySQL groups types into numeric, string, and date/time families.

4.1 Numeric Types

Common numeric types

Type	Use for	Notes
INT	Whole numbers	-2.1B to 2.1B; use UNSIGNED to double positive range
BIGINT	Very large integers	e.g. counts, IDs at scale
TINYINT	Tiny ints / booleans	TINYINT(1) often used as BOOLEAN
DECIMAL(p,s)	Money, exact values	DECIMAL(10,2) = 8 digits + 2 decimals
FLOAT / DOUBLE	Approximate reals	Avoid for money (rounding errors)

4.2 String Types

Common string types

Type	Use for	Notes
CHAR(n)	Fixed-length	Always n chars (padded); good for codes
VARCHAR(n)	Variable-length text	Most common; n = max length
TEXT	Long text	Articles, descriptions
ENUM(...)	Fixed set of values	e.g. status ENUM('Active','Closed')

4.3 Date & Time Types

Date & time types

Type	Format	Example
DATE	YYYY-MM-DD	2024-06-15
DATETIME	YYYY-MM-DD HH:MM:SS	2024-06-15 14:30:00
TIMESTAMP	as above, UTC-aware	auto-updates possible
TIME	HH:MM:SS	14:30:00
YEAR	YYYY	2024

4.4 Basic Syntax Rules

- SQL keywords are **case-insensitive** (**SELECT** = **select**) but UPPERCASE is conventional.
- Every statement ends with a **semicolon ;**.
- String and date literals go in **single quotes**: **'Mumbai'**, **'2024-06-15'**.

- Comments: `--` single line, `#` also single line, or `/*` multi-line `*/`.
- Identifiers with spaces/keywords use **backticks**: `order` .

A fully-commented query

```
-- Anatomy of a statement
SELECT name, city      -- columns to return
FROM   customers      -- source table
WHERE  city = 'Mumbai' -- filter
ORDER  BY name         -- sort
LIMIT 10;             -- cap rows
```

Practice Questions — Chapter 4

- Q1.** Which data type would you choose for: a product price, a mobile number, a person's date of birth, an order status of Pending/Shipped/Delivered? Justify each.
- Q2.** Why should money be stored as `DECIMAL` and not `FLOAT`?
- Q3.** What is the difference between `CHAR(10)` and `VARCHAR(10)`?
- Q4.** Write a single-line and a multi-line comment in SQL.
- Q5.** Rewrite this without errors: `select name city from Customer where City='Delhi'` (note the missing comma and quote style).

5. DDL — Creating and Altering Tables

DDL statements define and change the *structure* of your database. The core commands are **CREATE**, **ALTER**, **DROP**, **TRUNCATE** and **RENAME**.

5.1 CREATE TABLE

Setup Script — CREATE with column-level constraints

```
CREATE TABLE employees (  
    emp_id      INT PRIMARY KEY AUTO_INCREMENT,  
    name        VARCHAR(60) NOT NULL,  
    department  VARCHAR(40) DEFAULT 'General',  
    salary      DECIMAL(10,2),  
    hire_date   DATE,  
    email       VARCHAR(100) UNIQUE  
);
```

5.2 ALTER TABLE

ALTER changes an existing table without dropping it.

Common ALTER operations

```
ALTER TABLE employees ADD COLUMN phone VARCHAR(15);      -- add  
ALTER TABLE employees MODIFY COLUMN salary DECIMAL(12,2); -- change type  
ALTER TABLE employees CHANGE phone contact VARCHAR(15); -- rename+type  
ALTER TABLE employees DROP COLUMN contact;               -- remove  
ALTER TABLE employees ADD CONSTRAINT chk_sal CHECK (salary >= 0);
```

5.3 DROP vs TRUNCATE vs DELETE

A very common interview question!

Command	Removes	Structure kept?	Type	Rollback?
DELETE	Rows (with WHERE)	Yes	DML	Yes
TRUNCATE	All rows, fast	Yes	DDL	No
DROP	The whole table	No	DDL	No

✓ **Key Point:** **TRUNCATE** also **resets AUTO_INCREMENT** back to 1, whereas **DELETE** does not.

5.4 Renaming and Copying

Rename & copy

```
RENAME TABLE employees TO staff;  
CREATE TABLE staff_backup AS SELECT * FROM staff; -- copy structure + data  
CREATE TABLE staff_empty LIKE staff;              -- copy structure only
```

Practice Questions — Chapter 5

- Q1.** Create a table `book(book_id, title, author, price, published_on)` with an auto-incrementing primary key and a NOT NULL title.
- Q2.** Add a `stock INT DEFAULT 0` column to `book`, then change `price` to `DECIMAL(8,2)`.
- Q3.** Explain, with examples, the difference between DELETE, TRUNCATE and DROP.
- Q4.** Make a structural-only copy of `book` called `book_archive`.
- Q5.** Rename `book` to `library_book`, then drop the `library_book` table.

6. Constraints, Primary & Foreign Keys

Constraints are rules enforced by MySQL that keep your data valid. If an **INSERT** or **UPDATE** would break a constraint, MySQL rejects it.

6.1 The Six Constraints

The constraint toolbox

Constraint	Guarantees
NOT NULL	Column must have a value
UNIQUE	No two rows share the same value
PRIMARY KEY	UNIQUE + NOT NULL; identifies the row
FOREIGN KEY	Value must exist in the referenced table
CHECK	Value must satisfy a condition (MySQL 8.0.16+)
DEFAULT	Value used when none is supplied

6.2 Defining Constraints

Setup Script — table & column constraints together

```
CREATE TABLE departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(40) NOT NULL UNIQUE  
);  
  
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(60) NOT NULL,  
    salary DECIMAL(10,2) CHECK (salary > 0),  
    dept_id INT,  
    CONSTRAINT fk_dept FOREIGN KEY (dept_id)  
        REFERENCES departments(dept_id)  
        ON DELETE SET NULL ON UPDATE CASCADE  
);
```

6.3 Referential Actions

When a parent row is deleted/updated, the FK can react:

- **CASCADE** — apply the same change to child rows (delete the orders too).
- **SET NULL** — set the child FK to NULL.
- **RESTRICT / NO ACTION** — block the change if children exist (the default).

6.4 Composite Keys

Composite primary key

```
-- A junction table for the M:N 'students take courses'  
CREATE TABLE enrollments (  
    student_id INT,  
    course_id INT,  
    grade CHAR(2),  
    PRIMARY KEY (student_id, course_id) -- composite PK
```

```
);
```

Practice Questions — Chapter 6

- Q1.** Create a `categories(category_id, category_name UNIQUE)` and a `products(product_id, name, price CHECK>0, category_id FK)` schema.
- Q2.** Which single constraint is equivalent to combining UNIQUE and NOT NULL?
- Q3.** Explain ON DELETE CASCADE vs ON DELETE SET NULL with an orders/customers example.
- Q4.** Design a junction table for a M:N relationship between `actors` and `movies`. What is its primary key?
- Q5.** Try to insert a product with `price = -50` into a table with `CHECK (price > 0)`. What happens and why?

PART III

Querying Data

7. DML — INSERT, UPDATE, DELETE

DML (Data Manipulation Language) changes the *data* inside tables. The three workhorses are **INSERT** (add), **UPDATE** (change) and **DELETE** (remove).

7.1 INSERT

Setup Script — three ways to INSERT

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY AUTO_INCREMENT,  
    name       VARCHAR(50) NOT NULL,  
    price      DECIMAL(8,2),  
    stock      INT DEFAULT 0  
);  
  
-- 1) Full row, columns implied by position  
INSERT INTO products VALUES (1, 'Pen', 10.00, 500);  
  
-- 2) Named columns (recommended — order-independent)  
INSERT INTO products (name, price, stock) VALUES ('Notebook', 45.00, 200);  
  
-- 3) Multiple rows in one statement  
INSERT INTO products (name, price) VALUES  
    ('Eraser', 5.00), ('Marker', 25.00), ('Stapler', 120.00);
```

✓ **Key Point:** Always prefer the **named-column** form. It keeps working even if the table gains new columns later, and it is self-documenting.

7.2 UPDATE

UPDATE examples

```
-- Raise every price by 10%  
UPDATE products SET price = price * 1.10;  
  
-- Targeted update — ALWAYS use WHERE unless you mean 'all rows'  
UPDATE products SET stock = 1000 WHERE name = 'Pen';  
  
-- Update multiple columns at once  
UPDATE products SET price = 30, stock = 300 WHERE product_id = 4;
```

✓ **Key Point:** An **UPDATE** or **DELETE** without a **WHERE** clause affects every row. MySQL's *safe-update

mode* blocks this by default in Workbench — a good habit to keep.

7.3 DELETE

DELETE examples

```
DELETE FROM products WHERE stock = 0;    -- remove out-of-stock items
DELETE FROM products;                    -- remove ALL rows (structure stays)
```

7.4 INSERT ... SELECT and REPLACE

Bulk & upsert patterns

```
-- Copy selected rows into an archive table
INSERT INTO products_archive (name,price)
SELECT name, price FROM products WHERE price > 100;

-- Insert or overwrite by primary key
REPLACE INTO products (product_id,name,price) VALUES (1,'Gel Pen',15);
```

Practice Questions — Chapter 7

- Q1.** Using the **products** setup above, insert three more products of your choice in a single statement.
- Q2.** Increase the **stock** of every product priced below 50 by 100 units.
- Q3.** Change the name of product_id 2 to 'Spiral Notebook' and set its price to 60.
- Q4.** Delete all products whose price is greater than 100.
- Q5.** Write an **INSERT ... SELECT** that copies all products with stock > 200 into a table **bestsellers(name, price)**.

8. SELECT and the Core Clauses

SELECT is the command you will use most as a data analyst. This chapter introduces the shared **company dataset** used through the querying chapters, then the core clauses: **SELECT, FROM, WHERE**.

Setup Script — run this once; reused in Chapters 7–15

```
-- ===== Shared 'company' dataset for Chapters 7-15 =====
CREATE TABLE departments (
    dept_id    INT PRIMARY KEY,
    dept_name  VARCHAR(30) NOT NULL
);
INSERT INTO departments VALUES
(10,'Sales'),(20,'Engineering'),(30,'Support'),(40,'Marketing');

CREATE TABLE employees (
    emp_id     INT PRIMARY KEY AUTO_INCREMENT,
    name       VARCHAR(40) NOT NULL,
```

```

dept_id    INT,
salary     DECIMAL(10,2),
commission DECIMAL(10,2),      -- may be NULL
hire_date  DATE,
city       VARCHAR(30),
FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
INSERT INTO employees (name,dept_id,salary,commission,hire_date,city) VALUES
('Aarav', 10, 55000, 5000, '2021-03-15', 'Mumbai'),
('Diya', 10, 48000, 3000, '2021-07-01', 'Surat'),
('Kabir', 20, 82000, NULL, '2020-01-20', 'Delhi'),
('Anaya', 20, 76000, NULL, '2022-05-11', 'Delhi'),
('Vivaan', 30, 45000, NULL, '2023-02-18', 'Kochi'),
('Ishita', 30, 47000, 2000, '2022-09-30', 'Mumbai'),
('Arjun', 40, 60000, 4000, '2021-11-05', 'Pune'),
('Saarvi', 40, 52000, NULL, '2023-06-22', 'Chennai'),
('Rohan', NULL, 40000, NULL, '2023-08-01', 'Jaipur'); -- not yet assigned

```

8.1 Logical Order of a Query

You *write* a query as SELECT...FROM...WHERE..., but MySQL *executes* the clauses in a different order. Understanding this explains many beginner errors.

Logical execution order — memorise this!

Step	Clause	What it does
1	FROM / JOIN	Choose and combine tables
2	WHERE	Filter individual rows
3	GROUP BY	Form groups
4	HAVING	Filter groups
5	SELECT	Pick/compute columns
6	ORDER BY	Sort the result
7	LIMIT	Cap the rows

✓ **Key Point:** Because **SELECT** runs *after* **WHERE**, you **cannot** use a column alias defined in **SELECT** inside a **WHERE** clause. You can, however, use it in **ORDER BY**.

8.2 Selecting Columns

Projection & aliases

```

SELECT * FROM employees;           -- all columns
SELECT name, salary FROM employees; -- specific columns
SELECT name, salary * 12 AS annual_salary
FROM employees;                   -- computed column + alias

```

Result:

name	annual_salary
Aarav	660000.00
Diya	576000.00
...	...

8.3 Filtering with WHERE

WHERE examples

```
SELECT name, salary, city FROM employees  
WHERE salary > 50000;
```

```
SELECT name FROM employees WHERE city = 'Delhi';
```

```
SELECT name, hire_date FROM employees  
WHERE hire_date >= '2022-01-01';
```

Practice Questions — Chapter 8

- Q1.** Select the `name` and `city` of every employee.
- Q2.** Show `name` and a computed `monthly_take_home` equal to `salary/12` (aliased).
- Q3.** List employees who earn more than 50,000.
- Q4.** List employees hired on or after 1 Jan 2022.
- Q5.** Explain why `SELECT name, salary*12 AS annual FROM employees WHERE annual > 600000` fails, and fix it.

9. Operators in MySQL

Operators build the conditions in **WHERE** and the expressions in **SELECT**. MySQL provides arithmetic, comparison, logical, and special operators.

9.1 Arithmetic & Comparison

Arithmetic and comparison operators

Category	Operators
Arithmetic	+ - * / % (MOD)
Comparison	= <> (or !=) > < >= <=

9.2 Logical Operators — AND, OR, NOT

Combining conditions

```
SELECT name, salary, city FROM employees
WHERE salary > 45000 AND city = 'Mumbai';
```

```
SELECT name FROM employees
WHERE city = 'Delhi' OR city = 'Pune';
```

```
SELECT name FROM employees
WHERE NOT city = 'Delhi';
```

✓ **Key Point:** **AND** is evaluated before **OR**. Use **parentheses** to make intent explicit: `WHERE dept_id = 10 AND (city='Mumbai' OR city='Surat')`.

9.3 Special Operators

- **BETWEEN a AND b** — inclusive range: `salary BETWEEN 45000 AND 60000`.
- **IN (...)** — matches any value in a list: `city IN ('Mumbai','Delhi')`.
- **LIKE** — pattern match: `%` = any string, `_` = one char.
- **IS NULL / IS NOT NULL** — the **only** correct way to test for NULL.

BETWEEN, IN, LIKE, IS NULL

```
SELECT name, salary FROM employees WHERE salary BETWEEN 45000 AND 60000;
SELECT name FROM employees WHERE city IN ('Mumbai','Delhi');
SELECT name FROM employees WHERE name LIKE 'A%';      -- starts with A
SELECT name FROM employees WHERE name LIKE '_a%';     -- 2nd letter is a
SELECT name FROM employees WHERE commission IS NULL;
```

Practice Questions — Chapter 9

- Q1.** List employees earning between 45,000 and 60,000 inclusive.
- Q2.** List employees in Mumbai, Delhi or Pune using a single **IN** clause.

- Q3.** Find every employee whose name ends with the letter 'a'.
- Q4.** Find employees whose name contains the letter 'r' anywhere.
- Q5.** List employees who have a NULL commission and work in Support (dept_id 30).

10. Dealing with NULL Values

NULL means *unknown / missing*, not zero and not an empty string. It behaves specially in comparisons and calculations, so it deserves its own chapter.

10.1 The Golden Rules of NULL

- Any arithmetic with NULL yields NULL: `5000 + NULL → NULL`.
- `= NULL` is never true. Use `IS NULL` / `IS NOT NULL`.
- Aggregate functions (except `COUNT(*)`) ignore NULLs.
- NULL sorts first with `ORDER BY ... ASC` in MySQL.

Testing for NULL correctly

```
SELECT name, commission FROM employees WHERE commission IS NULL;
SELECT name, commission FROM employees WHERE commission IS NOT NULL;
```

10.2 Handling NULL — IFNULL, COALESCE, NULLIF

Setup uses the company dataset from Chapter 8

```
-- Treat missing commission as 0
SELECT name, IFNULL(commission, 0) AS commission FROM employees;

-- COALESCE returns the first non-NULL argument
SELECT name, COALESCE(commission, 0) AS commission FROM employees;

-- Total pay = salary + commission, safely
SELECT name, salary + IFNULL(commission,0) AS total_pay FROM employees;

-- NULLIF(a,b) returns NULL if a = b (useful to avoid divide-by-zero)
SELECT 100 / NULLIF(0,0) AS safe_div; -- NULL instead of an error
```

Result:

name	total_pay
Aarav	60000.00
Kabir	82000.00
...	...

✓ **Key Point:** **COALESCE** is the ANSI-standard, multi-argument version of **IFNULL**. Prefer **COALESCE** when you may have several fallback columns.

Practice Questions — Chapter 10

- Q1.** List every employee and their commission, showing 0 where it is NULL.
- Q2.** Compute each employee's total pay (salary + commission) treating NULL commission as 0.
- Q3.** How many employees have a NULL commission? (Hint: COUNT with a condition.)
- Q4.** Explain why `WHERE commission = NULL` returns no rows.
- Q5.** Use `NULLIF` to safely divide `salary` by `commission` without a divide-by-zero error.

11. Sorting, DISTINCT & LIMIT

These clauses shape the final result set: **ORDER BY** sorts, **DISTINCT** removes duplicates, and **LIMIT** caps the number of rows.

11.1 ORDER BY

Sorting results

```
SELECT name, salary FROM employees ORDER BY salary DESC;      -- highest first
SELECT name, city FROM employees ORDER BY city ASC, name;     -- multi-column
SELECT name, salary*12 AS annual FROM employees ORDER BY annual DESC; -- by alias
```

11.2 DISTINCT

Removing duplicates

```
SELECT DISTINCT city FROM employees;      -- unique cities
SELECT DISTINCT dept_id, city FROM employees; -- unique combinations
```

11.3 LIMIT and Pagination

Top-N and pagination

```
SELECT name, salary FROM employees ORDER BY salary DESC LIMIT 3;      -- top 3
SELECT name FROM employees ORDER BY name LIMIT 5 OFFSET 5;            -- rows 6-10
SELECT name FROM employees ORDER BY name LIMIT 5, 5;                  -- same, short form
```

✓ **Key Point:** To get a reliable *"Top N"* you must combine **ORDER BY** with **LIMIT**. Without **ORDER BY**, the rows returned by **LIMIT** are arbitrary.

Practice Questions — Chapter 11

- Q1.** List all employees sorted by salary from highest to lowest.
- Q2.** Show the distinct cities employees live in.

Q3. Find the top 3 highest-paid employees (name and salary).

Q4. Sort employees by city ascending, then by salary descending within each city.

Q5. Using LIMIT with OFFSET, return the 4th and 5th employees when ordered alphabetically by name.

PART IV

Summarising Data

12. Aggregate Functions & GROUP BY

Aggregate functions collapse many rows into a single summary value. Combined with **GROUP BY**, they answer questions like *"total sales per city"* or *"average salary per department"* — the heart of analytics.

Setup Script — run before Chapters 12–13

```
-- ===== 'sales' dataset for Chapters 12-13 =====
CREATE TABLE sales (
  sale_id INT PRIMARY KEY AUTO_INCREMENT,
  region VARCHAR(20),
  product VARCHAR(30),
  amount DECIMAL(10,2),
  qty INT,
  sale_date DATE
);
INSERT INTO sales (region,product,amount,qty,sale_date) VALUES
('North','Laptop', 60000, 1,'2024-01-05'),
('North','Mouse', 1200, 4,'2024-01-09'),
('South','Laptop', 58000, 1,'2024-02-11'),
('South','Keyboard',2500, 3,'2024-02-18'),
('East','Laptop', 61000, 1,'2024-03-02'),
('East','Mouse', 1500, 5,'2024-03-15'),
('West','Monitor',12000, 2,'2024-03-21'),
('North','Laptop', 59000, 1,'2024-04-04'),
('South','Mouse', 1300, 6,'2024-04-19'),
('West','Keyboard',2800, 2,'2024-05-01');
```

12.1 The Five Aggregate Functions

The core aggregates

Function	Returns
COUNT()	Number of rows / non-NULL values
SUM()	Total of a numeric column
AVG()	Average (ignores NULLs)
MIN()	Smallest value
MAX()	Largest value

Whole-table aggregation

```
SELECT COUNT(*) AS total_sales,
       SUM(amount) AS revenue,
       AVG(amount) AS avg_sale,
       MIN(amount) AS smallest,
       MAX(amount) AS largest
FROM sales;
```

Result:

total_sales	revenue	avg_sale	smallest	largest
10	259300.00	25930.00	1200.00	61000.00

12.2 GROUP BY

GROUP BY splits rows into groups; the aggregate is then computed **per group**.

Revenue per region

```
-- Revenue and number of sales per region
SELECT region, COUNT(*) AS sales, SUM(amount) AS revenue
FROM sales
GROUP BY region
ORDER BY revenue DESC;
```

Result:

region	sales	revenue
North	3	120200.00
East	2	62500.00
South	3	61800.00
West	2	14800.00

✓ **Key Point: Rule:** every column in **SELECT** that is not inside an aggregate function **must** appear in the **GROUP BY**. This is the single most common GROUP BY error.

12.3 Grouping by Multiple Columns

Two-level grouping

```
SELECT region, product, SUM(qty) AS units_sold
FROM sales
GROUP BY region, product
ORDER BY region, units_sold DESC;
```

Practice Questions — Chapter 12

- Q1.** Count the total number of sales and the total revenue across all rows.
- Q2.** Show the total quantity (**SUM(qty)**) sold per product.
- Q3.** Show the average sale amount per region, rounded to whole rupees.
- Q4.** For each region, show the largest single sale (**MAX(amount)**).
- Q5.** Group by region and product to show units sold for each combination.

13. The HAVING Clause

WHERE filters **rows** *before* grouping. **HAVING** filters **groups** *after* aggregation. Whenever your filter uses an aggregate (SUM, COUNT...), you need HAVING.

Filtering groups with HAVING

```
-- Regions whose total revenue exceeds 60,000
SELECT region, SUM(amount) AS revenue
FROM sales
GROUP BY region
HAVING SUM(amount) > 60000
ORDER BY revenue DESC;
```

Result:

region	revenue
North	120200.00
East	62500.00
South	61800.00

13.1 WHERE and HAVING Together

The classic WHERE + HAVING combo

```
-- Only 'Laptop' sales, grouped by region, keep regions with 2+ laptop sales
SELECT region, COUNT(*) AS laptop_sales, SUM(amount) AS revenue
FROM sales
WHERE product = 'Laptop'      -- row filter (before grouping)
GROUP BY region
HAVING COUNT(*) >= 2          -- group filter (after grouping)
ORDER BY revenue DESC;
```

Result:

region	laptop_sales	revenue
North	2	119000.00

WHERE vs HAVING — a favourite interview question

	WHERE	HAVING
Filters	Rows	Groups
Runs	Before GROUP BY	After GROUP BY
Can use aggregates?	No	Yes

Practice Questions — Chapter 13

- Q1.** Show products whose total revenue (SUM(amount)) is greater than 5,000.
- Q2.** Show regions that recorded more than 2 sales.
- Q3.** Among 'Mouse' sales only, show regions where total quantity sold exceeds 4.
- Q4.** Explain in one line the difference between WHERE and HAVING.
- Q5.** Show products with an average sale amount above 10,000, ordered highest first.

14. Built-in Functions (String, Numeric, Date)

MySQL ships with hundreds of scalar functions that transform values row-by-row. These are essential for **cleaning** and **reshaping** data.

Setup Script — run before Chapter 14

```
-- ===== 'people' dataset for Chapter 14 =====
CREATE TABLE people (
  id          INT PRIMARY KEY AUTO_INCREMENT,
  full_name   VARCHAR(50),
  email       VARCHAR(60),
  salary      DECIMAL(10,2),
  joined_on   DATE
);
INSERT INTO people (full_name,email,salary,joined_on) VALUES
(' aarav sharma ', 'AARAV@Mail.com', 55250.75, '2021-03-15'),
('diya patel',    'diya@Mail.com', 48999.20, '2022-07-01'),
('kabir singh',   'kabir@mail.com', 82500.00, '2020-01-20');
```

14.1 String Functions

Handy string functions

Function	Purpose	Example → Result
UPPER / LOWER	Change case	UPPER('abc') → ABC
LENGTH	Length in bytes	LENGTH('abc') → 3
CONCAT	Join strings	CONCAT('a','b') → ab
SUBSTRING	Extract part	SUBSTRING('MySQL',1,2) → My
TRIM	Remove spaces	TRIM(' hi ') → hi
REPLACE	Swap text	REPLACE('a-b','-','+') → a+b
LEFT / RIGHT	First/last n chars	LEFT('MySQL',2) → My

Cleaning names & emails

```
SELECT id,
  TRIM(full_name)           AS cleaned,
  UPPER(LEFT(TRIM(full_name),1)) AS initial,
  LOWER(email)             AS email_norm,
  SUBSTRING_INDEX(email,'@',-1) AS domain
FROM people;
```

14.2 Numeric Functions

Numeric functions

Function	Example → Result
ROUND(x, d)	ROUND(55250.75, 0) → 55251
CEIL / FLOOR	CEIL(4.2) → 5, FLOOR(4.8) → 4
ABS(x)	ABS(-5) → 5
MOD(a, b)	MOD(10,3) → 1
POWER(a, b)	POWER(2,3) → 8

14.3 Date & Time Functions

Date functions

Function	Purpose
NOW() / CURDATE()	Current datetime / date
YEAR() MONTH() DAY()	Extract parts of a date
DATEDIFF(a,b)	Days between two dates
DATE_ADD / DATE_SUB	Add/subtract an interval
DATE_FORMAT(d,fmt)	Format a date as text

Working with dates

```
SELECT full_name,
       YEAR(joined_on)           AS join_year,
       DATEDIFF(CURDATE(), joined_on) AS days_service,
       DATE_FORMAT(joined_on, '%d-%b-%Y') AS pretty_date,
       DATE_ADD(joined_on, INTERVAL 1 YEAR) AS first_anniversary
FROM people;
```

Practice Questions — Chapter 14

- Q1.** Return each person's name in Title style spacing removed (TRIM) and fully uppercase.
- Q2.** Extract the email **domain** (text after the @) for every person.
- Q3.** Round each salary to the nearest whole rupee and also to the nearest hundred.
- Q4.** Show how many days each person has been with the company (from `joined_on` to today).
- Q5.** Format `joined_on` as `15 March 2021` using `DATE_FORMAT`.

15. The CASE Operator & Control Flow

CASE adds **if-then-else** logic inside a query — perfect for bucketing values, creating labels, and conditional aggregation.

Searched CASE — bucketing salaries

```
-- Reuse the 'employees' dataset from Chapter 8
SELECT name, salary,
       CASE
         WHEN salary >= 70000 THEN 'High'
         WHEN salary >= 50000 THEN 'Medium'
         ELSE 'Entry'
       END AS salary_band
FROM employees
ORDER BY salary DESC;
```

Result:

name	salary	salary_band
Kabir	82000.00	High
Arjun	60000.00	Medium
Vivaan	45000.00	Entry

15.1 Simple vs Searched CASE

Simple CASE

```
-- Simple CASE compares one expression to values
SELECT name,
       CASE dept_id
         WHEN 10 THEN 'Sales'
         WHEN 20 THEN 'Engineering'
         ELSE 'Other'
       END AS dept_label
FROM employees;
```

15.2 Conditional Aggregation (a superpower)

Pivoting with CASE + SUM

```
-- Count employees per salary band in ONE row using CASE inside SUM
SELECT
  SUM(CASE WHEN salary >= 70000 THEN 1 ELSE 0 END) AS high,
  SUM(CASE WHEN salary BETWEEN 50000 AND 69999 THEN 1 ELSE 0 END) AS medium,
  SUM(CASE WHEN salary < 50000 THEN 1 ELSE 0 END) AS entry
FROM employees;
```

✓ **Key Point:** CASE ... END is an **expression**, so it can appear anywhere a value can: in SELECT, WHERE, ORDER BY, and inside aggregate functions.

Practice Questions — Chapter 15

- Q1.** Label each employee as 'Senior' if hired before 2022, else 'Recent'.
- Q2.** Create a `salary_band` column: High (≥70k), Medium (50–70k), Entry (<50k).
- Q3.** Use a simple CASE to turn dept_id 10/20/30/40 into readable department names.
- Q4.** Using conditional aggregation, count how many employees fall in each salary band in a single row.
- Q5.** Show employees ordered so that Mumbai staff appear first (Hint: `ORDER BY CASE WHEN city='Mumbai' THEN 0 ELSE 1 END`).

PART V

Combining Data

16. Joins in MySQL

Joins combine rows from two or more tables based on a related column. They are the reason relational databases are so powerful — and a top interview topic.

Setup Script — run before Chapters 16–20

```
-- ===== 'shop' dataset for Chapters 16-20 =====
CREATE TABLE customers (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(40), city VARCHAR(30));
INSERT INTO customers (name,city) VALUES
('Aarav', 'Mumbai'), ('Diya', 'Surat'), ('Kabir', 'Delhi'),
('Anaya', 'Hyderabad'), ('Vivaan', 'Kochi');

CREATE TABLE orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT, amount DECIMAL(10,2), order_date DATE,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id));
INSERT INTO orders (customer_id,amount,order_date) VALUES
(1, 2500, '2024-01-10'), (1, 1800, '2024-02-14'),
(2, 3200, '2024-01-22'), (3, 900, '2024-03-05'),
(3, 4100, '2024-03-19'), (1, 1500, '2024-04-02');
-- Note: Anaya(4) and Vivaan(5) have NO orders.
```

16.1 The Join Types

Join types at a glance

Join	Returns
INNER JOIN	Only matching rows in both tables
LEFT JOIN	All left rows + matches (NULLs where none)
RIGHT JOIN	All right rows + matches
CROSS JOIN	Every combination (Cartesian product)
SELF JOIN	A table joined to itself

16.2 INNER JOIN

Customers with their orders (matches only)

```
SELECT c.name, o.amount, o.order_date
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
ORDER BY c.name;
```

Result:

name	amount	order_date
Aarav	2500.00	2024-01-10

Aarav	1800.00	2024-02-14
Diya	3200.00	2024-01-22
Kabir	900.00	2024-03-05

✓ **Key Point:** Because Anaya and Vivaan have no orders, an INNER JOIN **excludes** them. To keep them, use a LEFT JOIN.

16.3 LEFT JOIN & Finding Unmatched Rows

LEFT JOIN + the anti-join pattern

```
-- Every customer, even those with no orders
SELECT c.name, o.amount
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;

-- Customers who have NEVER ordered (anti-join)
SELECT c.name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Result of the anti-join

name
Anaya
Vivaan

16.4 Joins with Aggregation

The join-group-by workhorse of analytics

```
-- Total spent by each customer (0 for non-buyers)
SELECT c.name, COUNT(o.order_id) AS orders,
       IFNULL(SUM(o.amount),0) AS total_spent
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
ORDER BY total_spent DESC;
```

16.5 Self Join

A table joined to itself

```
-- Employees and their managers (same table)
-- Assumes an employees(emp_id,name,manager_id) table
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```


Practice Questions — Chapter 16

- Q1.** List each customer's name alongside every order amount (INNER JOIN).
- Q2.** Show all customers and their total number of orders, including those with zero (LEFT JOIN).
- Q3.** Find the names of customers who have never placed an order.
- Q4.** Compute total revenue per city by joining customers and orders and grouping by city.
- Q5.** Show the single highest order amount together with the customer's name.

17. Set Operations: UNION, INTERSECT, EXCEPT

Set operators stack the results of two **SELECTs** **vertically**. Both queries must return the **same number of columns** with compatible types.

Setup Script — two customer lists

```
-- ===== dataset for Chapter 17 =====
CREATE TABLE online_customers (name VARCHAR(40));
CREATE TABLE store_customers (name VARCHAR(40));
INSERT INTO online_customers VALUES ('Aarav'),('Diya'),('Kabir'),('Anaya');
INSERT INTO store_customers VALUES ('Kabir'),('Anaya'),('Rohan'),('Myra');
```

17.1 UNION and UNION ALL

UNION removes duplicates; UNION ALL keeps them

```
-- All distinct customers across both channels
SELECT name FROM online_customers
UNION
SELECT name FROM store_customers;

-- Keep duplicates (faster, no de-dup)
SELECT name FROM online_customers
UNION ALL
SELECT name FROM store_customers;
```

UNION result (6 distinct names)

name
Aarav
Diya
Kabir
Anaya
Rohan
Myra

17.2 INTERSECT (MySQL 8.0.31+)

INTERSECT — common rows

```
-- Customers who shop BOTH online and in-store
SELECT name FROM online_customers
INTERSECT
SELECT name FROM store_customers;
```

Result:

name
Kabir
Anaya

17.3 EXCEPT (MySQL 8.0.31+)

EXCEPT — set difference

```
-- Customers who shop ONLY online (online minus store)
SELECT name FROM online_customers
EXCEPT
SELECT name FROM store_customers;
```

Result:

name
Aarav
Diya

✓ **Key Point:** On older MySQL versions without INTERSECT/EXCEPT, emulate them with **INNER JOIN** (intersect) and a **LEFT JOIN ... IS NULL** anti-join (except).

Practice Questions — Chapter 17

- Q1.** Produce one combined list of all distinct customer names across both tables.
- Q2.** Produce a list that keeps duplicates and explain when **UNION ALL** is preferable.
- Q3.** Find customers present in BOTH lists using INTERSECT.
- Q4.** Find customers who are online-only using EXCEPT.
- Q5.** Rewrite the INTERSECT query using an INNER JOIN instead (for older MySQL).

PART VI

Advanced Techniques

18. Subqueries

A **subquery** is a **SELECT** nested inside another query. It lets you use the result of one query as input to another — for filtering, computing, or comparing.

Setup Script — run before Chapters 18–20

```
-- ===== 'staff' dataset for Chapters 18-20 =====  
CREATE TABLE staff (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(40), dept VARCHAR(20), salary DECIMAL(10,2));  
INSERT INTO staff (name,dept,salary) VALUES  
( 'Aarav', 'Sales', 55000), ( 'Diya', 'Sales', 48000),  
( 'Kabin', 'Tech', 82000), ( 'Anaya', 'Tech', 76000),  
( 'Vivaan', 'HR', 45000), ( 'Ishita', 'HR', 47000),  
( 'Arjun', 'Sales', 60000), ( 'Saanvi', 'Tech', 52000);
```

18.1 Scalar Subquery (returns one value)

Compare against a single computed value

```
-- Everyone earning above the company average  
SELECT name, salary  
FROM staff  
WHERE salary > (SELECT AVG(salary) FROM staff)  
ORDER BY salary DESC;
```

Result:

name	salary
Kabin	82000.00
Anaya	76000.00
Arjun	60000.00

18.2 Subquery with IN (returns a list)

Filter by a set produced by a subquery

```
-- Staff in departments that have someone earning 80k+  
SELECT name, dept FROM staff  
WHERE dept IN (SELECT dept FROM staff WHERE salary >= 80000);
```

18.3 Correlated Subquery

A **correlated** subquery references the outer query and is evaluated once per outer row — powerful but slower.

Correlated subquery — per-group comparison

```
-- Each employee earning above their OWN department's average
SELECT s1.name, s1.dept, s1.salary
FROM staff s1
WHERE s1.salary > (
    SELECT AVG(s2.salary) FROM staff s2
    WHERE s2.dept = s1.dept      -- correlation
);
```

18.4 Subquery in FROM (derived table) & EXISTS

Derived tables and EXISTS

```
-- Derived table: average salary per dept, then filter
SELECT dept, avg_sal
FROM (SELECT dept, AVG(salary) AS avg_sal FROM staff GROUP BY dept) d
WHERE avg_sal > 50000;

-- EXISTS: departments that have at least one 80k earner
SELECT DISTINCT dept FROM staff s1
WHERE EXISTS (SELECT 1 FROM staff s2
    WHERE s2.dept=s1.dept AND s2.salary>=80000);
```

Practice Questions — Chapter 18

- Q1.** List staff earning more than the overall average salary.
- Q2.** Find staff whose salary equals the maximum salary in the whole table (use a scalar subquery).
- Q3.** Using IN, list staff who work in the same department as 'Kabir'.
- Q4.** Using a correlated subquery, find each person earning above their own department's average.
- Q5.** Using a derived table, list departments whose average salary exceeds 50,000.

19. Common Table Expressions (CTEs)

A **CTE** is a named temporary result defined with **WITH** that exists just for the following query. CTEs make complex queries **readable** and support **recursion**.

A CTE replaces a messy subquery

```
-- Basic CTE: department averages, then join back
WITH dept_avg AS (
    SELECT dept, AVG(salary) AS avg_sal
    FROM staff GROUP BY dept
)
SELECT s.name, s.dept, s.salary, d.avg_sal
FROM staff s
JOIN dept_avg d ON s.dept = d.dept
WHERE s.salary > d.avg_sal;
```

✓ **Key Point:** A CTE reads top-to-bottom like a paragraph, whereas nested subqueries read inside-out. For anything beyond one level of nesting, prefer a CTE.

19.1 Multiple CTEs

Chaining CTEs

```
WITH
  sales_dept AS (SELECT * FROM staff WHERE dept='Sales'),
  high_paid AS (SELECT * FROM sales_dept WHERE salary > 50000)
SELECT name, salary FROM high_paid ORDER BY salary DESC;
```

19.2 Recursive CTE

A **recursive CTE** repeatedly references itself — ideal for hierarchies (org charts, categories) and generating sequences.

Classic recursive number generator

```
-- Generate numbers 1..10
WITH RECURSIVE nums AS (
  SELECT 1 AS n                                -- anchor member
  UNION ALL
  SELECT n + 1 FROM nums WHERE n < 10 -- recursive member
)
SELECT n FROM nums;
```

Result:

n
1
2
3
...
10

Practice Questions — Chapter 19

- Q1.** Rewrite the 'above department average' query from Chapter 18 using a CTE.
- Q2.** Use two chained CTEs to list Tech staff earning above 60,000.
- Q3.** Write a recursive CTE that lists the numbers 1 to 20.
- Q4.** Write a recursive CTE that outputs the first 8 multiples of 5.
- Q5.** Explain one advantage of a CTE over a derived table (subquery in FROM).

20. Window Functions

Window functions perform calculations across a set of rows **related to the current row**, *without collapsing* them like GROUP BY does. They are the most powerful analytics feature in modern SQL.

The syntax is `function() OVER (PARTITION BY ... ORDER BY ...)`. **PARTITION BY** divides rows into groups; **ORDER BY** orders rows within each partition.

20.1 Ranking Functions

ROW_NUMBER vs RANK vs DENSE_RANK, per department

```
SELECT name, dept, salary,
       ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) AS rn,
       RANK()        OVER (PARTITION BY dept ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS dr
FROM staff;
```

Result:

name	dept	salary	rn	rnk	dr
Kabir	Tech	82000	1	1	1
Anaya	Tech	76000	2	2	2
Saanvi	Tech	52000	3	3	3
Arjun	Sales	60000	1	1	1

How the three ranking functions treat ties

Function	On a tie...
ROW_NUMBER()	Distinct numbers, arbitrary tie-break
RANK()	Same rank, then SKIPS (1,1,3)
DENSE_RANK()	Same rank, NO gap (1,1,2)

20.2 Aggregate Window Functions

Aggregates as windows — keep detail + summary together

```
-- Department total & average shown on EVERY row (no collapsing)
SELECT name, dept, salary,
       SUM(salary) OVER (PARTITION BY dept) AS dept_total,
       ROUND(AVG(salary) OVER (PARTITION BY dept),0) AS dept_avg,
       ROUND(100*salary/SUM(salary) OVER (PARTITION BY dept),1) AS pct_of_dept
FROM staff;
```

20.3 Running Totals & LAG / LEAD

Running totals and row-to-row comparison

```
-- Running total of salary ordered by id
SELECT name, salary,
       SUM(salary) OVER (ORDER BY id) AS running_total
FROM staff;

-- Compare each row to the previous one
```

```
SELECT name, salary,
       LAG(salary) OVER (ORDER BY salary) AS prev_salary,
       LEAD(salary) OVER (ORDER BY salary) AS next_salary
FROM staff;
```

✓ **Key Point:** Window functions run **after** WHERE/GROUP BY/HAVING but **before** ORDER BY of the outer query. To filter on a window result (e.g. rank ≤ 3), wrap it in a CTE or subquery, then filter outside.

20.4 Top-N per Group (a killer pattern)

Top-2 earners in each department

```
WITH ranked AS (
  SELECT name, dept, salary,
         ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) AS rn
  FROM staff
)
SELECT name, dept, salary FROM ranked WHERE rn <= 2;
```

Practice Questions — Chapter 20

- Q1.** Rank all staff by salary (highest = 1) using RANK().
- Q2.** Within each department, number staff from highest to lowest paid with ROW_NUMBER().
- Q3.** Show each person's salary as a percentage of their department's total.
- Q4.** Compute a running total of salaries ordered by id.
- Q5.** Return the single highest-paid employee **in each department** using a window function + CTE.

21. Views

A **view** is a stored query that behaves like a virtual table. It hides complexity, enforces consistent logic, and can restrict which columns users see.

Setup Script — creating and using a view

```
-- Build on the 'shop' dataset (customers, orders) from Chapter 16
CREATE VIEW customer_summary AS
SELECT c.customer_id, c.name, c.city,
       COUNT(o.order_id)      AS num_orders,
       IFNULL(SUM(o.amount),0) AS total_spent
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name, c.city;

-- Query it like a table
SELECT * FROM customer_summary WHERE total_spent > 3000;
```


21.1 Managing Views

View lifecycle

```
CREATE OR REPLACE VIEW customer_summary AS SELECT ...; -- redefine
SHOW FULL TABLES WHERE table_type = 'VIEW';         -- list views
DROP VIEW IF EXISTS customer_summary;                 -- remove
```

✓ **Key Point:** A view stores **the query, not the data** — it always reflects the latest data in the underlying tables. Views with a single table and no aggregation are often **updatable** (you can INSERT/UPDATE through them).

Practice Questions — Chapter 21

- Q1.** Create a view `high_value_orders` showing orders with amount above 2,000 together with the customer name.
- Q2.** Query your view to list only Mumbai customers.
- Q3.** Redefine the view using CREATE OR REPLACE to also include the order date.
- Q4.** List all views in the current database.
- Q5.** Explain one security benefit of exposing a view instead of the base table.

22. Stored Procedures & Functions

A **stored procedure** is a saved block of SQL you call by name, optionally passing parameters. It centralises business logic and reduces repetition. A **stored function** returns a single value and can be used inside queries.

22.1 Creating a Procedure

A parameterised procedure

```
DELIMITER //
CREATE PROCEDURE get_staff_by_dept(IN p_dept VARCHAR(20))
BEGIN
    SELECT name, salary
    FROM staff
    WHERE dept = p_dept
    ORDER BY salary DESC;
END //
DELIMITER ;

CALL get_staff_by_dept('Tech');
```

✓ **Key Point:** We change the statement terminator with `DELIMITER //` so the semicolons **inside** the procedure aren't treated as the end of the CREATE statement. Reset it to `;` afterwards.

22.2 IN / OUT Parameters

Returning a value via OUT

```
DELIMITER //
CREATE PROCEDURE count_in_dept(IN p_dept VARCHAR(20), OUT p_count INT)
BEGIN
    SELECT COUNT(*) INTO p_count FROM staff WHERE dept = p_dept;
END //
DELIMITER ;

CALL count_in_dept('Sales', @n);
SELECT @n AS sales_count;
```

22.3 A Stored Function

A reusable scalar function

```
DELIMITER //
CREATE FUNCTION annual_salary(monthly DECIMAL(10,2))
RETURNS DECIMAL(12,2) DETERMINISTIC
BEGIN
    RETURN monthly * 12;
END //
DELIMITER ;

SELECT name, annual_salary(salary) AS yearly FROM staff;
```

22.4 Control Flow inside Routines

- **Variables:** `DECLARE v INT DEFAULT 0;`
- **Conditionals:** `IF ... THEN ... ELSEIF ... ELSE ... END IF;`
- **Loops:** `WHILE ... DO ... END WHILE;`, `LOOP, REPEAT.`
- **Manage routines:** `SHOW PROCEDURE STATUS;`, `DROP PROCEDURE name;`.

Practice Questions — Chapter 22

- Q1.** Write a procedure `staff_above(IN p_sal DECIMAL)` that lists staff earning more than the given salary.
- Q2.** Write a procedure with an OUT parameter that returns the average salary of a given department.
- Q3.** Create a stored function `tax(amount)` that returns 10% of the amount, and use it in a SELECT.
- Q4.** Call your procedure from Q1 with the value 50000.
- Q5.** Explain the purpose of the `DELIMITER //` statement when defining a procedure.

23. Indexes & Query Optimization

As tables grow, **indexes** keep queries fast. An index is a sorted lookup structure (like a book's index) that lets MySQL find rows without scanning the whole table.

23.1 Creating Indexes

Index management

```
CREATE INDEX idx_staff_dept ON staff(dept);           -- single column
CREATE INDEX idx_staff_dept_sal ON staff(dept, salary); -- composite
CREATE UNIQUE INDEX idx_email ON people(email);       -- unique index
SHOW INDEX FROM staff;
DROP INDEX idx_staff_dept ON staff;
```

✓ **Key Point:** Index the columns you frequently **filter** (WHERE), **join** on, and **sort** by. Don't over-index: every index speeds reads but slows down INSERT/UPDATE and uses disk.

23.2 Reading EXPLAIN

Ask MySQL how it will run a query

```
EXPLAIN SELECT name FROM staff WHERE dept = 'Tech';
```

Key EXPLAIN 'type' values

type	Meaning (best → worst)
const / eq_ref	Primary-key lookup – excellent
ref	Uses a non-unique index – good
range	Index range scan – ok
ALL	Full table scan – consider an index

23.3 Optimization Checklist

- Select only the columns you need — avoid **SELECT *** in production.
- Filter early with **WHERE**; make sure filtered columns are indexed.
- Avoid functions on indexed columns in WHERE (e.g. **WHERE YEAR(d)=2024** can't use an index; use a range instead).
- Use **LIMIT** when you only need a few rows.
- Prefer **JOIN** over correlated subqueries for large datasets.

Practice Questions — Chapter 23

- Q1.** Create a composite index on **staff(dept, salary)** and explain when it helps.
- Q2.** Run EXPLAIN on a query filtering by **dept** before and after adding an index; describe the change in **type**.
- Q3.** Why can **WHERE YEAR(joined_on) = 2024** prevent index usage? Rewrite it as a range condition.
- Q4.** List three practical rules for deciding which columns to index.

Q5. Explain the trade-off of adding many indexes to a write-heavy table.

24. AI-Assisted SQL (ChatGPT, Gemini, SQLCoder)

The syllabus highlights **AI for SQL** — using tools like ChatGPT, Gemini and SQLCoder to generate queries, debug errors, optimise, and document SQL. AI is an accelerator, not a replacement for understanding.

24.1 Generating Queries from Plain English

Give the AI your **schema** and a clear request. The more context (table names, columns, sample rows), the better the output.

A good 'text-to-SQL' prompt

```
Prompt template:  
"Here is my MySQL schema:  
  staff(id, name, dept, salary)  
Write a query to find the top 2 highest-paid employees per  
department. Use MySQL 8 window functions."
```

24.2 Debugging & Optimising

- Paste the **exact error message** plus your query and ask for the fix and the reason.
- Ask **"why is this slow?"** along with the output of **EXPLAIN** for optimisation ideas.
- Ask the AI to **add comments** to a complex query for auto-documentation.
- Ask it to **rewrite** a nested subquery as a CTE for readability.

24.3 Always Verify

✓ **Key Point:** AI can **hallucinate** columns or use another dialect's syntax. Always: (1) run the query on a small dataset, (2) check row counts and totals against a known value, and (3) confirm it is valid **MySQL** syntax before trusting it.

Practice Questions — Chapter 24

- Q1.** Write a prompt (including a schema) that asks an AI to produce a monthly revenue report.
- Q2.** Take a query you wrote earlier and ask an AI to add explanatory comments; review the result.
- Q3.** Describe three checks you must perform on any AI-generated query before using it.
- Q4.** Explain why providing sample rows improves an AI's SQL output.
- Q5.** Give an example of a mistake (wrong column / wrong dialect) an AI might make and how you'd catch it.

PART VII

Practice Papers

10 Papers × 100 Marks

Each paper is fully self-contained: the Setup Script at the top creates every table and inserts every row needed to answer all 16 questions in that paper. Solve them in MySQL Workbench, then check your logic against the concepts in Parts I–VI.

Practice Paper 1: E-Commerce Sales Analysis (Amazon-style)

You are a Data Analyst at an online store. Analyse customers, products and orders.

Exam details

Time	Max Marks	Sections
3 Hours	100	A(20) + B(30) + C(40) + D(10)

Instructions: First run the Setup Script below to create and populate the tables. All questions in this paper are solved using these tables. Write standard **MySQL 8.0+** queries. Marks are shown against each question.

Setup Script — run this first (creates & inserts all data for this paper)

```
CREATE DATABASE IF NOT EXISTS pp1; USE pp1;
DROP TABLE IF EXISTS order_items, orders, products, customers, categories;

CREATE TABLE categories (
  category_id INT PRIMARY KEY, category_name VARCHAR(30));
INSERT INTO categories VALUES
(1,'Electronics'),(2,'Books'),(3,'Clothing'),(4,'Home');

CREATE TABLE customers (
  customer_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(40), city VARCHAR(30), signup_date DATE);
INSERT INTO customers (name,city,signup_date) VALUES
('Aarav','Mumbai','2023-01-10'),('Diya','Delhi','2023-02-15'),
('Kabir','Mumbai','2023-03-20'),('Anaya','Pune','2023-05-01'),
('Vivaan','Delhi','2023-06-12'),('Ishita','Surat','2023-08-09');

CREATE TABLE products (
  product_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(40), category_id INT, price DECIMAL(10,2), stock INT,
  FOREIGN KEY (category_id) REFERENCES categories(category_id));
INSERT INTO products (name,category_id,price,stock) VALUES
('Smartphone',1,24999,50),('Laptop',1,58999,20),('Earbuds',1,4999,120),
('SQL Book',2,650,80),('Novel',2,399,60),('T-Shirt',3,799,200),
('Jeans',3,1999,90),('Cookware',4,2499,40);

CREATE TABLE orders (
  order_id INT PRIMARY KEY AUTO_INCREMENT,
  customer_id INT, order_date DATE, status VARCHAR(15),
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id));
INSERT INTO orders (customer_id,order_date,status) VALUES
(1,'2024-01-05','Delivered'),(2,'2024-01-18','Delivered'),
(1,'2024-02-10','Shipped'),(3,'2024-02-22','Delivered'),
(4,'2024-03-03','Cancelled'),(2,'2024-03-19','Delivered'),
(5,'2024-04-01','Pending'),(1,'2024-04-25','Delivered');

CREATE TABLE order_items (
  order_id INT, product_id INT, quantity INT, unit_price DECIMAL(10,2),
  PRIMARY KEY(order_id,product_id),
  FOREIGN KEY (order_id) REFERENCES orders(order_id),
  FOREIGN KEY (product_id) REFERENCES products(product_id));
```

```
INSERT INTO order_items VALUES
(1,1,1,24999),(1,3,2,4999),(2,4,3,650),(2,5,1,399),
(3,2,1,58999),(4,6,4,799),(4,7,1,1999),(6,3,1,4999),
(6,8,2,2499),(8,1,1,24999),(8,4,2,650);
```

Section A — Concepts, DDL & DML (5 × 4 = 20 Marks)

Attempt all questions

- Q1. Add a new column `phone VARCHAR(15)` to the `customers` table. [4]
- Q2. Insert a new product: 'Headphones', category Electronics, price 1499, stock 75. [4]
- Q3. Update the status of order_id 7 from 'Pending' to 'Shipped'. [4]
- Q4. Write a query to delete all orders whose status is 'Cancelled'. [4]
- Q5. Name the primary key of `order_items` and explain why it is composite. [4]

Section B — SELECT, Filtering & Functions (5 × 6 = 30 Marks)

Attempt all questions

- Q1. List all products priced above 2,000, sorted from most to least expensive. [6]
- Q2. Show the names of customers who signed up in 2023 and live in Mumbai or Delhi. [6]
- Q3. Display each product's name and price with a 10% discounted price (aliased `sale_price`). [6]
- Q4. Show the distinct cities customers come from. [6]
- Q5. List the 3 cheapest products (name and price). [6]

Section C — Aggregation, Joins, Subqueries, CTE & Windows (5 × 8 = 40 Marks)

Attempt all questions

- Q1. Show each customer's name and their total number of orders (include customers with zero orders). [8]
- Q2. Compute total revenue (quantity × unit_price) per product, highest first. [8]
- Q3. For each category, show the number of products and the average price. [8]
- Q4. Find customers who have never placed an order. [8]
- Q5. Using a subquery, list products priced above the average product price. [8]

Section D — Practical Challenge (1 × 10 = 10 Marks)

Attempt the question

- Q1. Using a window function, rank products **within each category** by price (highest = 1) and return only the

top product in every category (name, category, price). **[10]**

Practice Paper 2: Human Resources & Payroll

You are an HR analyst examining departments, employees and salaries.

Exam details

Time	Max Marks	Sections
3 Hours	100	A(20) + B(30) + C(40) + D(10)

Instructions: First run the Setup Script below to create and populate the tables. All questions in this paper are solved using these tables. Write standard **MySQL 8.0+** queries. Marks are shown against each question.

Setup Script — run this first (creates & inserts all data for this paper)

```
CREATE DATABASE IF NOT EXISTS pp2; USE pp2;
DROP TABLE IF EXISTS employees, departments;

CREATE TABLE departments (
  dept_id INT PRIMARY KEY, dept_name VARCHAR(30), location VARCHAR(30));
INSERT INTO departments VALUES
(10, 'Sales', 'Mumbai'), (20, 'Engineering', 'Bengaluru'),
(30, 'Support', 'Pune'), (40, 'Marketing', 'Delhi');

CREATE TABLE employees (
  emp_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(40), dept_id INT, salary DECIMAL(10,2),
  commission DECIMAL(10,2), hire_date DATE, manager_id INT,
  FOREIGN KEY (dept_id) REFERENCES departments(dept_id));
INSERT INTO employees (name, dept_id, salary, commission, hire_date, manager_id) VALUES
('Neha', 20, 120000, NULL, '2020-01-10', NULL),
('Raj', 10, 65000, 8000, '2021-03-15', 1),
('Priya', 10, 58000, 5000, '2021-06-01', 2),
('Sameer', 30, 52000, NULL, '2022-02-20', 1),
('Tara', 30, 47000, 2000, '2022-05-11', 4),
('Karan', 20, 88000, NULL, '2021-08-19', 1),
('Aisha', 10, 61000, 6000, '2023-01-05', 2),
('Rohan', 40, 54000, NULL, '2023-04-18', 1),
('Myra', 40, 49000, 3000, '2023-07-22', 8);
```

Section A — Concepts, DDL & DML (5 × 4 = 20 Marks)

Attempt all questions

- Q1.** Add a **CHECK** constraint idea: write an **ALTER** that ensures salary is always > 0. **[4]**
- Q2.** Insert a new employee 'Isha' in Engineering (dept 20), salary 70000, hired today, manager 1. **[4]**
- Q3.** Give every Support (dept 30) employee a 5% salary raise. **[4]**
- Q4.** Explain the difference between **DELETE** and **TRUNCATE** for the **employees** table. **[4]**
- Q5.** Which column is a self-referencing foreign key here, and what does it represent? **[4]**

Section B — SELECT, Filtering & Functions (5 × 6 = 30 Marks)

Attempt all questions

- Q1.** List employees earning between 50,000 and 70,000, sorted by salary descending. [6]
- Q2.** Show each employee's name and total pay = salary + commission (treat NULL commission as 0). [6]
- Q3.** List employees hired after 1 Jan 2022 whose name starts with a vowel. [6]
- Q4.** Show employees who have no commission (NULL), with their department id. [6]
- Q5.** Display name and a `tenure_years` column = whole years since hire_date. [6]

Section C — Aggregation, Joins, Subqueries, CTE & Windows (5 × 8 = 40 Marks)

Attempt all questions

- Q1.** Show the average, minimum and maximum salary per department (with department name). [8]
- Q2.** List departments having more than 2 employees (use GROUP BY + HAVING). [8]
- Q3.** For each employee, show their manager's name (self join). [8]
- Q4.** Using a correlated subquery, list employees earning above their own department's average. [8]
- Q5.** Show each employee, their salary, and the department total salary using a window function. [8]

Section D — Practical Challenge (1 × 10 = 10 Marks)

Attempt the question

- Q1.** Using RANK() over each department ordered by salary descending, return the top-2 earners per department (name, department, salary, rank). [10]

Practice Paper 3: Library Management System

You manage a library's books, members and loan records.

Exam details

Time	Max Marks	Sections
3 Hours	100	A(20) + B(30) + C(40) + D(10)

Instructions: First run the Setup Script below to create and populate the tables. All questions in this paper are solved using these tables. Write standard **MySQL 8.0+** queries. Marks are shown against each question.

Setup Script — run this first (creates & inserts all data for this paper)

```
CREATE DATABASE IF NOT EXISTS pp3; USE pp3;
DROP TABLE IF EXISTS loans, books, members;

CREATE TABLE members (
  member_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(40), city VARCHAR(30), join_date DATE);
INSERT INTO members (name,city,join_date) VALUES
('Aarav','Mumbai','2023-01-15'),('Diya','Delhi','2023-02-20'),
('Kabir','Mumbai','2023-04-10'),('Anaya','Pune','2023-06-25'),
('Vivaan','Delhi','2023-09-05');

CREATE TABLE books (
  book_id INT PRIMARY KEY AUTO_INCREMENT,
  title VARCHAR(50), author VARCHAR(40), genre VARCHAR(20),
  copies INT, price DECIMAL(8,2));
INSERT INTO books (title,author,genre,copies,price) VALUES
('SQL Basics','R. Rao','Technical',5,650),
('Data Science 101','M. Iyer','Technical',3,899),
('The Lost City','A. Khan','Fiction',4,399),
('Mystery Manor','A. Khan','Fiction',2,349),
('World History','P. Nair','History',6,550),
('Python Crash','S. Bose','Technical',0,750);

CREATE TABLE loans (
  loan_id INT PRIMARY KEY AUTO_INCREMENT,
  member_id INT, book_id INT, loan_date DATE, return_date DATE,
  FOREIGN KEY (member_id) REFERENCES members(member_id),
  FOREIGN KEY (book_id) REFERENCES books(book_id));
INSERT INTO loans (member_id,book_id,loan_date,return_date) VALUES
(1,1,'2024-01-05','2024-01-20'),(1,3,'2024-02-10',NULL),
(2,2,'2024-02-15','2024-03-01'),(3,5,'2024-03-08','2024-03-25'),
(3,1,'2024-04-01',NULL),(4,3,'2024-04-12','2024-04-28'),
(1,5,'2024-05-02',NULL);
```

Section A — Concepts, DDL & DML (5 × 4 = 20 Marks)

Attempt all questions

Q1. Add a column `email VARCHAR(60)` to `members`. [4]

Q2. Insert a new book: 'Advanced SQL' by 'R. Rao', Technical, 4 copies, price 1200. [4]

- Q3.** Update the return_date of loan_id 2 to today's date. [4]
- Q4.** Delete all books that currently have 0 copies. [4]
- Q5.** Write a query to create a structural copy of the books table named books_archive. [4]

Section B — SELECT, Filtering & Functions (5 × 6 = 30 Marks)

Attempt all questions

- Q1.** List all Technical books with more than 3 copies, sorted by price. [6]
- Q2.** Show book titles and authors for the 'Fiction' genre only. [6]
- Q3.** Display each book's title and a discounted price at 15% off. [6]
- Q4.** List members who joined in the first half of 2023 (Jan–Jun). [6]
- Q5.** Show the distinct genres available in the library. [6]

Section C — Aggregation, Joins, Subqueries, CTE & Windows (5 × 8 = 40 Marks)

Attempt all questions

- Q1.** Show how many books each author has written (count by author). [8]
- Q2.** For each member, show their name and total number of loans (include members with none). [8]
- Q3.** List currently borrowed books (return_date IS NULL) with the member and book title. [8]
- Q4.** Find genres whose average book price exceeds 500 (GROUP BY + HAVING). [8]
- Q5.** Using a subquery, list books priced above the average book price. [8]

Section D — Practical Challenge (1 × 10 = 10 Marks)

Attempt the question

- Q1.** Write a query using a CTE that lists each member with their loan count, then returns only the member(s) with the highest number of loans. [10]

Practice Paper 4: Banking & Transactions

You analyse customers, accounts and transactions for a retail bank.

Exam details

Time	Max Marks	Sections
3 Hours	100	A(20) + B(30) + C(40) + D(10)

Instructions: First run the Setup Script below to create and populate the tables. All questions in this paper are solved using these tables. Write standard **MySQL 8.0+** queries. Marks are shown against each question.

Setup Script — run this first (creates & inserts all data for this paper)

```
CREATE DATABASE IF NOT EXISTS pp4; USE pp4;
DROP TABLE IF EXISTS transactions, accounts, bank_customers;

CREATE TABLE bank_customers (
  cust_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(40), city VARCHAR(30), kyc_done VARCHAR(3));
INSERT INTO bank_customers (name,city,kyc_done) VALUES
('Aarav','Mumbai','Yes'),('Diya','Delhi','Yes'),('Kabir','Pune','No'),
('Anaya','Mumbai','Yes'),('Vivaan','Surat','No');

CREATE TABLE accounts (
  acc_id INT PRIMARY KEY AUTO_INCREMENT,
  cust_id INT, acc_type VARCHAR(10), balance DECIMAL(12,2), opened DATE,
  FOREIGN KEY (cust_id) REFERENCES bank_customers(cust_id));
INSERT INTO accounts (cust_id,acc_type,balance,opened) VALUES
(1,'Savings',85000,'2022-05-10'),(1,'Current',150000,'2023-01-15'),
(2,'Savings',42000,'2022-08-20'),(3,'Savings',12000,'2023-03-05'),
(4,'Current',230000,'2021-11-11'),(4,'Savings',67000,'2023-06-30');

CREATE TABLE transactions (
  txn_id INT PRIMARY KEY AUTO_INCREMENT,
  acc_id INT, txn_type VARCHAR(10), amount DECIMAL(12,2), txn_date DATE,
  FOREIGN KEY (acc_id) REFERENCES accounts(acc_id));
INSERT INTO transactions (acc_id,txn_type,amount,txn_date) VALUES
(1,'Credit',20000,'2024-01-10'),(1,'Debit',5000,'2024-01-15'),
(2,'Debit',30000,'2024-02-01'),(3,'Credit',5000,'2024-02-12'),
(4,'Credit',50000,'2024-03-03'),(4,'Debit',12000,'2024-03-20'),
(5,'Debit',8000,'2024-04-05'),(1,'Credit',10000,'2024-04-18');
```

Section A — Concepts, DDL & DML (5 × 4 = 20 Marks)

Attempt all questions

- Q1.** Add a **CHECK** that balance cannot be negative (write the ALTER TABLE). **[4]**
- Q2.** Insert a new Savings account for cust_id 2 with balance 30000 opened today. **[4]**
- Q3.** Update KYC status of Kabir (cust_id 3) to 'Yes'. **[4]**
- Q4.** Delete all transactions of type 'Debit' below 10,000. **[4]**
- Q5.** Explain the foreign-key relationship between **accounts** and **transactions**. **[4]**

Section B — SELECT, Filtering & Functions (5 × 6 = 30 Marks)

Attempt all questions

- Q1.** List all Savings accounts with a balance above 50,000, sorted descending. [6]
- Q2.** Show customers whose KYC is not done. [6]
- Q3.** Display each account's id, type and balance with a projected balance after 4% interest. [6]
- Q4.** List accounts opened before 2023, showing account id and opened date. [6]
- Q5.** Show the distinct account types in the bank. [6]

Section C — Aggregation, Joins, Subqueries, CTE & Windows (5 × 8 = 40 Marks)

Attempt all questions

- Q1.** Show total balance held per customer (name + sum of balances). [8]
- Q2.** For each account, show the number of transactions and total credited amount. [8]
- Q3.** List customers who hold more than one account (GROUP BY + HAVING). [8]
- Q4.** Find the account(s) with the maximum balance using a subquery. [8]
- Q5.** Show each transaction with a running total of amount per account (window function). [8]

Section D — Practical Challenge (1 × 10 = 10 Marks)

Attempt the question

- Q1.** Using a CTE, compute each account's net movement (total Credit – total Debit) and return the accounts with a positive net movement, ordered largest first. [10]

Practice Paper 5: Hospital & Patient Records

You are a health-data analyst working with doctors, patients and appointments.

Exam details

Time	Max Marks	Sections
3 Hours	100	A(20) + B(30) + C(40) + D(10)

Instructions: First run the Setup Script below to create and populate the tables. All questions in this paper are solved using these tables. Write standard **MySQL 8.0+** queries. Marks are shown against each question.

Setup Script — run this first (creates & inserts all data for this paper)

```
CREATE DATABASE IF NOT EXISTS pp5; USE pp5;
DROP TABLE IF EXISTS appointments, patients, doctors;

CREATE TABLE doctors (
  doctor_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(40), specialty VARCHAR(25), fee DECIMAL(8,2));
INSERT INTO doctors (name,specialty,fee) VALUES
('Dr. Rao','Cardiology',800),('Dr. Iyer','Dermatology',500),
('Dr. Khan','Orthopedics',700),('Dr. Nair','Cardiology',900),
('Dr. Bose','Pediatrics',400);

CREATE TABLE patients (
  patient_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(40), age INT, gender CHAR(1), city VARCHAR(30));
INSERT INTO patients (name,age,gender,city) VALUES
('Aarav',34,'M','Mumbai'),('Diya',28,'F','Delhi'),
('Kabir',45,'M','Pune'),('Anaya',52,'F','Mumbai'),
('Vivaan',7,'M','Surat'),('Ishita',63,'F','Delhi');

CREATE TABLE appointments (
  appt_id INT PRIMARY KEY AUTO_INCREMENT,
  patient_id INT, doctor_id INT, appt_date DATE, status VARCHAR(12),
  FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
  FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id));
INSERT INTO appointments (patient_id,doctor_id,appt_date,status) VALUES
(1,1,'2024-01-08','Completed'),(2,2,'2024-01-15','Completed'),
(3,3,'2024-02-02','Cancelled'),(4,1,'2024-02-19','Completed'),
(5,5,'2024-03-05','Completed'),(1,4,'2024-03-22','Completed'),
(6,4,'2024-04-10','Pending'),(2,2,'2024-04-27','Completed');
```

Section A — Concepts, DDL & DML (5 × 4 = 20 Marks)

Attempt all questions

- Q1.** Add a column **phone VARCHAR(15)** to **patients**. [4]
- Q2.** Insert a new doctor 'Dr. Sharma', Neurology, fee 1000. [4]
- Q3.** Update the status of **appt_id** 7 to 'Completed'. [4]
- Q4.** Delete all appointments with status 'Cancelled'. [4]

Q5. Identify all foreign keys in the `appointments` table. [4]

Section B — SELECT, Filtering & Functions (5 × 6 = 30 Marks)

Attempt all questions

- Q1.** List patients older than 40, sorted by age descending. [6]
- Q2.** Show doctors in Cardiology with a fee above 800. [6]
- Q3.** Display each doctor's name and fee with an 18% GST-added `total_fee`. [6]
- Q4.** List female patients from Mumbai or Delhi. [6]
- Q5.** Categorise patients as 'Child' (<18), 'Adult' (18–59) or 'Senior' (60+) using CASE. [6]

Section C — Aggregation, Joins, Subqueries, CTE & Windows (5 × 8 = 40 Marks)

Attempt all questions

- Q1.** Count the number of appointments per doctor (with doctor name). [8]
- Q2.** Show total revenue per specialty from Completed appointments (join + group). [8]
- Q3.** List doctors who have more than one completed appointment (HAVING). [8]
- Q4.** For each patient, show their number of appointments (include patients with none). [8]
- Q5.** Using a subquery, list doctors whose fee is above the average doctor fee. [8]

Section D — Practical Challenge (1 × 10 = 10 Marks)

Attempt the question

- Q1.** Using a window function, for each specialty rank doctors by fee (highest = 1) and return the top-earning doctor per specialty along with their total number of appointments. [10]

Practice Paper 6: School: Students, Courses & Grades

You analyse a school's students, the courses they enrol in, and their marks.

Exam details

Time	Max Marks	Sections
3 Hours	100	A(20) + B(30) + C(40) + D(10)

Instructions: First run the Setup Script below to create and populate the tables. All questions in this paper are solved using these tables. Write standard **MySQL 8.0+** queries. Marks are shown against each question.

Setup Script — run this first (creates & inserts all data for this paper)

```
CREATE DATABASE IF NOT EXISTS pp6; USE pp6;
DROP TABLE IF EXISTS enrollments, courses, students;

CREATE TABLE students (
  student_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(40), grade_level INT, city VARCHAR(30));
INSERT INTO students (name,grade_level,city) VALUES
('Aarav',10,'Mumbai'),('Diya',10,'Delhi'),('Kabir',11,'Mumbai'),
('Anaya',11,'Pune'),('Vivaan',12,'Delhi'),('Ishita',12,'Surat');

CREATE TABLE courses (
  course_id INT PRIMARY KEY AUTO_INCREMENT,
  course_name VARCHAR(30), credits INT, teacher VARCHAR(30));
INSERT INTO courses (course_name,credits,teacher) VALUES
('Mathematics',4,'Mr. Rao'),('Physics',4,'Ms. Iyer'),
('English',3,'Mrs. Khan'),('Computer Science',3,'Mr. Bose');

CREATE TABLE enrollments (
  student_id INT, course_id INT, marks INT,
  PRIMARY KEY(student_id,course_id),
  FOREIGN KEY (student_id) REFERENCES students(student_id),
  FOREIGN KEY (course_id) REFERENCES courses(course_id));
INSERT INTO enrollments VALUES
(1,1,88),(1,2,76),(1,4,92),(2,1,64),(2,3,80),
(3,2,90),(3,4,85),(4,1,55),(4,3,70),(5,2,95),
(5,4,60),(6,1,73),(6,2,82);
```

Section A — Concepts, DDL & DML (5 × 4 = 20 Marks)

Attempt all questions

- Q1.** Add a column `email VARCHAR(60)` to `students`. [4]
- Q2.** Insert a new course 'Chemistry', 4 credits, teacher 'Dr. Nair'. [4]
- Q3.** Increase the marks of student_id 4 in course_id 1 by 5. [4]
- Q4.** Explain why `enrollments` uses a composite primary key. [4]
- Q5.** Write a query to delete enrollments where marks are below 40. [4]

Section B — SELECT, Filtering & Functions (5 × 6 = 30 Marks)

Attempt all questions

- Q1.** List students in grade 12, sorted by name. [6]
- Q2.** Show all enrollment rows with marks of 80 or above. [6]
- Q3.** Display each course name and its **weighted** value = credits × 10. [6]
- Q4.** List students from Mumbai or Pune. [6]
- Q5.** Label each enrollment's marks as 'A' (≥85), 'B' (70–84), 'C' (<70) using CASE. [6]

Section C — Aggregation, Joins, Subqueries, CTE & Windows (5 × 8 = 40 Marks)

Attempt all questions

- Q1.** Show the average marks per course (with course name). [8]
- Q2.** For each student, show their total marks across all courses (join + group). [8]
- Q3.** List courses where the average marks exceed 80 (HAVING). [8]
- Q4.** Show each student's name and how many courses they are enrolled in. [8]
- Q5.** Using a subquery, find students who scored above the overall average mark in any course. [8]

Section D — Practical Challenge (1 × 10 = 10 Marks)

Attempt the question

- Q1.** Using RANK() partitioned by course and ordered by marks descending, return the top scorer in each course (student name, course name, marks). [10]

Practice Paper 7: Food Delivery Analytics (Swiggy-style)

You are a Data Analyst at a food-delivery app studying restaurants, orders and delivery.

Exam details

Time	Max Marks	Sections
3 Hours	100	A(20) + B(30) + C(40) + D(10)

Instructions: First run the Setup Script below to create and populate the tables. All questions in this paper are solved using these tables. Write standard **MySQL 8.0+** queries. Marks are shown against each question.

Setup Script — run this first (creates & inserts all data for this paper)

```
CREATE DATABASE IF NOT EXISTS pp7; USE pp7;
DROP TABLE IF EXISTS deliveries, food_orders, restaurants;

CREATE TABLE restaurants (
  rest_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(40), cuisine VARCHAR(20), city VARCHAR(30), rating DECIMAL(2,1));
INSERT INTO restaurants (name,cuisine,city,rating) VALUES
('Spice Hub','Indian','Mumbai',4.5),('Pizza Point','Italian','Delhi',4.2),
('Wok Star','Chinese','Mumbai',3.9),('Curry Leaf','Indian','Pune',4.7),
('Burger Barn','American','Delhi',4.0);

CREATE TABLE food_orders (
  order_id INT PRIMARY KEY AUTO_INCREMENT,
  rest_id INT, customer VARCHAR(40), amount DECIMAL(8,2), order_date DATE,
  FOREIGN KEY (rest_id) REFERENCES restaurants(rest_id));
INSERT INTO food_orders (rest_id,customer,amount,order_date) VALUES
(1,'Aarav',450,'2024-01-05'),(1,'Diya',300,'2024-01-09'),
(2,'Kabir',650,'2024-01-15'),(3,'Anaya',220,'2024-02-01'),
(4,'Vivaan',540,'2024-02-11'),(1,'Ishita',380,'2024-02-20'),
(2,'Aarav',720,'2024-03-03'),(4,'Diya',600,'2024-03-18'),
(5,'Kabir',280,'2024-04-02');

CREATE TABLE deliveries (
  delivery_id INT PRIMARY KEY AUTO_INCREMENT,
  order_id INT, agent VARCHAR(30), minutes INT,
  FOREIGN KEY (order_id) REFERENCES food_orders(order_id));
INSERT INTO deliveries (order_id,agent,minutes) VALUES
(1,'Ravi',28),(2,'Sunil',35),(3,'Ravi',42),(4,'Meena',25),
(5,'Sunil',30),(6,'Meena',38),(7,'Ravi',50),(8,'Sunil',33);
```

Section A — Concepts, DDL & DML (5 × 4 = 20 Marks)

Attempt all questions

- Q1.** Add a column `veg_only VARCHAR(3)` to `restaurants`. [4]
- Q2.** Insert a new restaurant 'Taco Town', Mexican, Pune, rating 4.1. [4]
- Q3.** Update the rating of 'Wok Star' to 4.1. [4]
- Q4.** Delete all `food_orders` below 250 rupees. [4]

Q5. State the relationship between `food_orders` and `deliveries`. [4]

Section B — SELECT, Filtering & Functions (5 × 6 = 30 Marks)

Attempt all questions

- Q1.** List Indian restaurants with a rating of 4.5 or higher. [6]
- Q2.** Show orders placed in February 2024, sorted by amount descending. [6]
- Q3.** Display each order's amount with a 5% platform fee added (aliased `total`). [6]
- Q4.** List restaurants in Mumbai or Delhi. [6]
- Q5.** Bucket orders as 'Large' (≥ 500) or 'Regular' (< 500) using CASE. [6]

Section C — Aggregation, Joins, Subqueries, CTE & Windows (5 × 8 = 40 Marks)

Attempt all questions

- Q1.** Show total revenue and number of orders per restaurant (name + aggregates). [8]
- Q2.** Show average delivery time (minutes) per delivery agent. [8]
- Q3.** List cuisines whose total order value exceeds 1,000 (join + HAVING). [8]
- Q4.** For each restaurant, show its order count (include those with zero orders). [8]
- Q5.** Using a subquery, find restaurants with a rating above the average rating. [8]

Section D — Practical Challenge (1 × 10 = 10 Marks)

Attempt the question

- Q1.** Using a window function, rank restaurants **within each city** by total revenue and return the top restaurant per city (name, city, total_revenue). [10]

Practice Paper 8: Video Streaming Platform (Netflix-style)

You analyse titles, users and viewing history for a streaming service.

Exam details

Time	Max Marks	Sections
3 Hours	100	A(20) + B(30) + C(40) + D(10)

Instructions: First run the Setup Script below to create and populate the tables. All questions in this paper are solved using these tables. Write standard **MySQL 8.0+** queries. Marks are shown against each question.

Setup Script — run this first (creates & inserts all data for this paper)

```
CREATE DATABASE IF NOT EXISTS pp8; USE pp8;
DROP TABLE IF EXISTS views, titles, users;

CREATE TABLE users (
  user_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(40), plan VARCHAR(10), country VARCHAR(20), joined DATE);
INSERT INTO users (name,plan,country,joined) VALUES
('Aarav','Premium','India','2023-01-10'),('Diya','Basic','India','2023-03-15'),
('Kabir','Standard','USA','2023-05-20'),('Anaya','Premium','UK','2023-07-01'),
('Vivaan','Basic','India','2023-09-12');

CREATE TABLE titles (
  title_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50), genre VARCHAR(20), release_year INT, duration_min INT);
INSERT INTO titles (name,genre,release_year,duration_min) VALUES
('The Last Case','Thriller',2022,128),('Laugh Out','Comedy',2021,95),
('Space Beyond','SciFi',2023,142),('Royal Court','Drama',2020,110),
('Fast Lane','Action',2023,118),('Cook It','Reality',2022,45);

CREATE TABLE views (
  view_id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT, title_id INT, watch_date DATE, minutes_watched INT,
  FOREIGN KEY (user_id) REFERENCES users(user_id),
  FOREIGN KEY (title_id) REFERENCES titles(title_id));
INSERT INTO views (user_id,title_id,watch_date,minutes_watched) VALUES
(1,1,'2024-01-05',128),(1,3,'2024-01-12',100),(2,2,'2024-01-20',95),
(3,5,'2024-02-02',118),(4,4,'2024-02-15',110),(1,5,'2024-02-28',60),
(2,6,'2024-03-08',45),(4,1,'2024-03-19',128),(5,3,'2024-04-01',142);
```

Section A — Concepts, DDL & DML (5 × 4 = 20 Marks)

Attempt all questions

- Q1.** Add a column `last_login DATE` to `users`. [4]
- Q2.** Insert a new title 'Deep Blue', Documentary, 2023, 88 minutes. [4]
- Q3.** Upgrade Diya's plan (user_id 2) from 'Basic' to 'Standard'. [4]
- Q4.** Delete all titles released before 2021. [4]
- Q5.** Name the two foreign keys in the `views` table. [4]

Section B — SELECT, Filtering & Functions (5 × 6 = 30 Marks)

Attempt all questions

- Q1.** List titles released in 2023 longer than 100 minutes, sorted by duration. [6]
- Q2.** Show users on the 'Premium' plan from India or UK. [6]
- Q3.** Display each title's name and its duration in hours (aliased, 2 decimals). [6]
- Q4.** List the distinct genres available. [6]
- Q5.** Label titles as 'Long' (≥120 min) or 'Short' (<120) using CASE. [6]

Section C — Aggregation, Joins, Subqueries, CTE & Windows (5 × 8 = 40 Marks)

Attempt all questions

- Q1.** Show total minutes watched per user (with user name). [8]
- Q2.** Show the number of views and total minutes per genre (join + group). [8]
- Q3.** List genres watched for more than 150 total minutes (HAVING). [8]
- Q4.** For each title, show how many times it was watched (include never-watched titles). [8]
- Q5.** Using a subquery, find titles longer than the average title duration. [8]

Section D — Practical Challenge (1 × 10 = 10 Marks)

Attempt the question

- Q1.** Using a CTE and a window function, compute total minutes watched per user, then return each user's single most-watched title (user name, title name, minutes_watched). [10]

Practice Paper 9: Airline Booking System

You analyse flights, passengers and bookings for an airline.

Exam details

Time	Max Marks	Sections
3 Hours	100	A(20) + B(30) + C(40) + D(10)

Instructions: First run the Setup Script below to create and populate the tables. All questions in this paper are solved using these tables. Write standard **MySQL 8.0+** queries. Marks are shown against each question.

Setup Script — run this first (creates & inserts all data for this paper)

```
CREATE DATABASE IF NOT EXISTS pp9; USE pp9;
DROP TABLE IF EXISTS bookings, flights, passengers;

CREATE TABLE flights (
  flight_id INT PRIMARY KEY AUTO_INCREMENT,
  flight_no VARCHAR(8), origin VARCHAR(20), destination VARCHAR(20),
  fare DECIMAL(8,2), seats INT);
INSERT INTO flights (flight_no,origin,destination,fare,seats) VALUES
('AI101','Mumbai','Delhi',5500,180),('AI202','Delhi','Bengaluru',4800,160),
('AI303','Mumbai','Chennai',6200,150),('AI404','Delhi','Mumbai',5300,180),
('AI505','Pune','Delhi',4200,120);

CREATE TABLE passengers (
  passenger_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(40), city VARCHAR(20), tier VARCHAR(10));
INSERT INTO passengers (name,city,tier) VALUES
('Aarav','Mumbai','Gold'),('Diya','Delhi','Silver'),('Kabin','Pune','Gold'),
('Anaya','Mumbai','Platinum'),('Vivaan','Chennai','Silver'),('Ishita','Delhi','Gold');

CREATE TABLE bookings (
  booking_id INT PRIMARY KEY AUTO_INCREMENT,
  passenger_id INT, flight_id INT, booking_date DATE, amount DECIMAL(8,2),
  FOREIGN KEY (passenger_id) REFERENCES passengers(passenger_id),
  FOREIGN KEY (flight_id) REFERENCES flights(flight_id));
INSERT INTO bookings (passenger_id,flight_id,booking_date,amount) VALUES
(1,1,'2024-01-05',5500),(2,2,'2024-01-15',4800),(3,1,'2024-02-01',5500),
(4,3,'2024-02-12',6200),(1,4,'2024-02-25',5300),(5,2,'2024-03-08',4800),
(6,1,'2024-03-20',5500),(4,5,'2024-04-02',4200),(2,3,'2024-04-18',6200);
```

Section A — Concepts, DDL & DML (5 × 4 = 20 Marks)

Attempt all questions

- Q1.** Add a column **email** **VARCHAR(60)** to **passengers**. **[4]**
- Q2.** Insert a new flight 'AI606' from Bengaluru to Mumbai, fare 5000, 160 seats. **[4]**
- Q3.** Increase the fare of all Mumbai-origin flights by 10%. **[4]**
- Q4.** Delete bookings made before 1 Feb 2024. **[4]**
- Q5.** Explain the foreign-key relationships in the **bookings** table. **[4]**

Section B — SELECT, Filtering & Functions (5 × 6 = 30 Marks)

Attempt all questions

- Q1.** List flights with fare above 5,000, sorted by fare descending. [6]
- Q2.** Show Gold or Platinum tier passengers from Mumbai or Delhi. [6]
- Q3.** Display each flight with a **route** column formatted as 'Origin -> Destination' (use CONCAT). [6]
- Q4.** List distinct origin cities served. [6]
- Q5.** Label flights as 'Premium' (fare ≥ 6000) or 'Economy' otherwise using CASE. [6]

Section C — Aggregation, Joins, Subqueries, CTE & Windows (5 × 8 = 40 Marks)

Attempt all questions

- Q1.** Show total revenue and number of bookings per flight (with flight number). [8]
- Q2.** Show total amount spent per passenger tier (join + group). [8]
- Q3.** List flights booked more than once (HAVING COUNT > 1). [8]
- Q4.** For each passenger, show their number of bookings (include those with none). [8]
- Q5.** Using a subquery, list flights with a fare above the average fare. [8]

Section D — Practical Challenge (1 × 10 = 10 Marks)

Attempt the question

- Q1.** Using a window function, rank flights by total revenue and also show each flight's revenue as a percentage of the airline's grand total revenue. [10]

Practice Paper 10: Warehouse & Inventory Management

You manage suppliers, products and stock movements across a warehouse.

Exam details

Time	Max Marks	Sections
3 Hours	100	A(20) + B(30) + C(40) + D(10)

Instructions: First run the Setup Script below to create and populate the tables. All questions in this paper are solved using these tables. Write standard **MySQL 8.0+** queries. Marks are shown against each question.

Setup Script — run this first (creates & inserts all data for this paper)

```
CREATE DATABASE IF NOT EXISTS pp10; USE pp10;
DROP TABLE IF EXISTS stock_moves, inventory, suppliers;

CREATE TABLE suppliers (
  supplier_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(40), city VARCHAR(30), rating DECIMAL(2,1));
INSERT INTO suppliers (name,city,rating) VALUES
('TechWorld','Mumbai',4.5),('HomeLine','Delhi',4.1),
('FitGear','Pune',4.3),('PlayZone','Kolkata',3.8);

CREATE TABLE inventory (
  item_id INT PRIMARY KEY AUTO_INCREMENT,
  item_name VARCHAR(40), supplier_id INT, unit_cost DECIMAL(10,2), quantity INT,
  FOREIGN KEY (supplier_id) REFERENCES suppliers(supplier_id));
INSERT INTO inventory (item_name,supplier_id,unit_cost,quantity) VALUES
('Laptop',1,45000,25),('Mouse',1,450,300),('Kettle',2,899,80),
('Blender',2,1799,40),('Dumbbell',3,999,60),('Yoga Mat',3,499,120),
('Board Game',4,799,50),('RC Car',4,1499,0);

CREATE TABLE stock_moves (
  move_id INT PRIMARY KEY AUTO_INCREMENT,
  item_id INT, move_type VARCHAR(6), qty INT, move_date DATE,
  FOREIGN KEY (item_id) REFERENCES inventory(item_id));
INSERT INTO stock_moves (item_id,move_type,qty,move_date) VALUES
(1,'IN',10,'2024-01-05'),(1,'OUT',5,'2024-01-20'),(2,'OUT',50,'2024-02-01'),
(3,'IN',20,'2024-02-10'),(4,'OUT',8,'2024-03-02'),(5,'IN',15,'2024-03-15'),
(6,'OUT',30,'2024-04-01'),(2,'IN',100,'2024-04-12'),(1,'OUT',3,'2024-04-25');
```

Section A — Concepts, DDL & DML (5 × 4 = 20 Marks)

Attempt all questions

- Q1.** Add a **CHECK** ensuring quantity is never negative (write the ALTER TABLE). **[4]**
- Q2.** Insert a new item 'Monitor' from supplier 1, unit_cost 12000, quantity 30. **[4]**
- Q3.** Reduce the quantity of item_id 2 (Mouse) by 50. **[4]**
- Q4.** Delete all inventory items with quantity 0. **[4]**
- Q5.** Explain what happens to **inventory** rows if a referenced supplier is deleted with ON DELETE RESTRICT. **[4]**

Section B — SELECT, Filtering & Functions (5 × 6 = 30 Marks)

Attempt all questions

- Q1.** List items with `unit_cost` above 1,000, sorted descending. [6]
- Q2.** Show items supplied by suppliers located in Mumbai or Pune. [6]
- Q3.** Display each item's name and `stock_value = unit_cost × quantity`. [6]
- Q4.** List items whose quantity is below 50 (low stock). [6]
- Q5.** Label items as 'Out' (0), 'Low' (<50) or 'OK' (≥50) using CASE. [6]

Section C — Aggregation, Joins, Subqueries, CTE & Windows (5 × 8 = 40 Marks)

Attempt all questions

- Q1.** Show total stock value per supplier (name + sum of `unit_cost × quantity`). [8]
- Q2.** Count the number of stock moves per item (with item name). [8]
- Q3.** List suppliers whose average item cost exceeds 1,000 (HAVING). [8]
- Q4.** For each item, show total 'IN' quantity and total 'OUT' quantity (use conditional aggregation). [8]
- Q5.** Using a subquery, find items priced above the average `unit_cost`. [8]

Section D — Practical Challenge (1 × 10 = 10 Marks)

Attempt the question

- Q1.** Using a CTE, compute each item's net stock change (total IN – total OUT) from `stock_moves`, then list items whose net change is negative (i.e. net outflow), ordered most-negative first. [10]

APPENDIX A

Solutions to the 10 Practice Papers

Solutions — Practice Paper 1

E-Commerce Sales Analysis

Model answers below. Several questions have more than one correct approach; these are reference solutions in MySQL 8.0+ syntax.

Section A

A1

```
ALTER TABLE customers ADD COLUMN phone VARCHAR(15);
```

A2

```
INSERT INTO products (name,category_id,price,stock)
VALUES ('Headphones',1,1499,75);
```

A3

```
UPDATE orders SET status='Shipped' WHERE order_id=7;
```

A4

```
DELETE FROM orders WHERE status='Cancelled';
```

A5

```
-- PK of order_items is (order_id, product_id): composite,
-- because one order can contain many products and one product
-- can appear in many orders; the pair is unique per line item.
```

Section B

B1

```
SELECT name, price FROM products
WHERE price > 2000 ORDER BY price DESC;
```

B2

```
SELECT name FROM customers
WHERE YEAR(signup_date)=2023 AND city IN ('Mumbai','Delhi');
```

B3

```
SELECT name, price, ROUND(price*0.90,2) AS sale_price FROM products;
```

B4

```
SELECT DISTINCT city FROM customers;
```

B5

```
SELECT name, price FROM products ORDER BY price ASC LIMIT 3;
```

Section C

C1

```
SELECT c.name, COUNT(o.order_id) AS orders
FROM customers c
LEFT JOIN orders o ON c.customer_id=o.customer_id
GROUP BY c.customer_id, c.name;
```

C2

```
SELECT p.name, SUM(oi.quantity*oi.unit_price) AS revenue
FROM order_items oi
JOIN products p ON oi.product_id=p.product_id
GROUP BY p.product_id, p.name
ORDER BY revenue DESC;
```

C3

```
SELECT c.category_name, COUNT(*) AS num_products,
       ROUND(AVG(p.price),2) AS avg_price
FROM categories c
JOIN products p ON c.category_id=p.category_id
GROUP BY c.category_name;
```

C4

```
SELECT c.name FROM customers c
LEFT JOIN orders o ON c.customer_id=o.customer_id
WHERE o.order_id IS NULL;
```

C5

```
SELECT name, price FROM products
WHERE price > (SELECT AVG(price) FROM products);
```

Section D

D

```
WITH ranked AS (
  SELECT p.name, c.category_name, p.price,
         ROW_NUMBER() OVER (PARTITION BY p.category_id
                             ORDER BY p.price DESC) AS rn
  FROM products p JOIN categories c ON p.category_id=c.category_id
)
SELECT name, category_name, price FROM ranked WHERE rn=1;
```

Solutions — Practice Paper 2

Human Resources & Payroll

Model answers below. Several questions have more than one correct approach; these are reference solutions in MySQL 8.0+ syntax.

Section A

A1

```
ALTER TABLE employees ADD CONSTRAINT chk_sal CHECK (salary > 0);
```

A2

```
INSERT INTO employees (name,dept_id,salary,commission,hire_date,manager_id)
VALUES ('Isha',20,70000,NULL,CURDATE(),1);
```

A3

```
UPDATE employees SET salary = salary*1.05 WHERE dept_id=30;
```

A4

```
-- DELETE removes chosen rows (WHERE), is DML, can be rolled back,
-- keeps AUTO_INCREMENT. TRUNCATE removes ALL rows, is DDL, faster,
-- cannot be rolled back and resets AUTO_INCREMENT.
```

A5

```
-- manager_id is a self-referencing FK to employees(emp_id);
-- it stores each employee's manager.
```

Section B

B1

```
SELECT name, salary FROM employees
WHERE salary BETWEEN 50000 AND 70000 ORDER BY salary DESC;
```

B2

```
SELECT name, salary + IFNULL(commission,0) AS total_pay FROM employees;
```

B3

```
SELECT name, hire_date FROM employees
WHERE hire_date > '2022-01-01'
AND LEFT(name,1) IN ('A','E','I','O','U');
```

B4

```
SELECT name, dept_id FROM employees WHERE commission IS NULL;
```

B5

```
SELECT name, TIMESTAMPDIF(YEAR, hire_date, CURDATE()) AS tenure_years  
FROM employees;
```

Section C

C1

```
SELECT d.dept_name, ROUND(AVG(e.salary),0) AS avg_sal,  
       MIN(e.salary) AS min_sal, MAX(e.salary) AS max_sal  
FROM departments d JOIN employees e ON d.dept_id=e.dept_id  
GROUP BY d.dept_name;
```

C2

```
SELECT d.dept_name, COUNT(*) AS headcount  
FROM departments d JOIN employees e ON d.dept_id=e.dept_id  
GROUP BY d.dept_name HAVING COUNT(*) > 2;
```

C3

```
SELECT e.name AS employee, m.name AS manager  
FROM employees e LEFT JOIN employees m ON e.manager_id=m.emp_id;
```

C4

```
SELECT e1.name, e1.dept_id, e1.salary FROM employees e1  
WHERE e1.salary > (SELECT AVG(e2.salary) FROM employees e2  
                  WHERE e2.dept_id=e1.dept_id);
```

C5

```
SELECT name, salary,  
       SUM(salary) OVER (PARTITION BY dept_id) AS dept_total  
FROM employees;
```

Section D

D

```
WITH r AS (  
  SELECT e.name, d.dept_name, e.salary,  
         RANK() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC) AS rnk  
  FROM employees e JOIN departments d ON e.dept_id=d.dept_id  
)  
SELECT name, dept_name, salary, rnk FROM r WHERE rnk <= 2;
```

Solutions — Practice Paper 3

Library Management System

Model answers below. Several questions have more than one correct approach; these are reference solutions in MySQL 8.0+ syntax.

Section A

A1

```
ALTER TABLE members ADD COLUMN email VARCHAR(60);
```

A2

```
INSERT INTO books (title,author,genre,copies,price)
VALUES ('Advanced SQL','R. Rao','Technical',4,1200);
```

A3

```
UPDATE loans SET return_date=CURDATE() WHERE loan_id=2;
```

A4

```
DELETE FROM books WHERE copies=0;
```

A5

```
CREATE TABLE books_archive LIKE books;
```

Section B

B1

```
SELECT * FROM books WHERE genre='Technical' AND copies>3 ORDER BY price;
```

B2

```
SELECT title, author FROM books WHERE genre='Fiction';
```

B3

```
SELECT title, ROUND(price*0.85,2) AS discounted FROM books;
```

B4

```
SELECT name FROM members WHERE join_date BETWEEN '2023-01-01' AND '2023-06-30';
```

B5

```
SELECT DISTINCT genre FROM books;
```


Section C

C1

```
SELECT author, COUNT(*) AS num_books FROM books GROUP BY author;
```

C2

```
SELECT m.name, COUNT(l.loan_id) AS loans
FROM members m LEFT JOIN loans l ON m.member_id=l.member_id
GROUP BY m.member_id, m.name;
```

C3

```
SELECT m.name, b.title FROM loans l
JOIN members m ON l.member_id=m.member_id
JOIN books b ON l.book_id=b.book_id
WHERE l.return_date IS NULL;
```

C4

```
SELECT genre, AVG(price) AS avg_price FROM books
GROUP BY genre HAVING AVG(price) > 500;
```

C5

```
SELECT title, price FROM books
WHERE price > (SELECT AVG(price) FROM books);
```

Section D

D

```
WITH lc AS (
  SELECT m.name, COUNT(l.loan_id) AS loans
  FROM members m LEFT JOIN loans l ON m.member_id=l.member_id
  GROUP BY m.member_id, m.name
)
SELECT name, loans FROM lc
WHERE loans = (SELECT MAX(loans) FROM lc);
```

Solutions — Practice Paper 4

Banking & Transactions

Model answers below. Several questions have more than one correct approach; these are reference solutions in MySQL 8.0+ syntax.

Section A

A1

```
ALTER TABLE accounts ADD CONSTRAINT chk_bal CHECK (balance >= 0);
```

A2

```
INSERT INTO accounts (cust_id,acc_type,balance,opened)
VALUES (2,'Savings',30000,CURDATE());
```

A3

```
UPDATE bank_customers SET kyc_done='Yes' WHERE cust_id=3;
```

A4

```
DELETE FROM transactions WHERE txn_type='Debit' AND amount < 10000;
```

A5

```
-- transactions.acc_id is a FK referencing accounts.acc_id:
-- every transaction must belong to an existing account.
```

Section B

B1

```
SELECT * FROM accounts WHERE acc_type='Savings' AND balance>50000
ORDER BY balance DESC;
```

B2

```
SELECT name FROM bank_customers WHERE kyc_done='No';
```

B3

```
SELECT acc_id, acc_type, balance,
       ROUND(balance*1.04,2) AS after_interest FROM accounts;
```

B4

```
SELECT acc_id, opened FROM accounts WHERE opened < '2023-01-01';
```

B5

```
SELECT DISTINCT acc_type FROM accounts;
```

Section C

C1

```
SELECT c.name, SUM(a.balance) AS total_balance
FROM bank_customers c JOIN accounts a ON c.cust_id=a.cust_id
GROUP BY c.cust_id, c.name;
```

C2

```
SELECT a.acc_id, COUNT(t.txn_id) AS num_txns,
       SUM(CASE WHEN t.txn_type='Credit' THEN t.amount ELSE 0 END) AS credited
FROM accounts a LEFT JOIN transactions t ON a.acc_id=t.acc_id
GROUP BY a.acc_id;
```

C3

```
SELECT c.name, COUNT(*) AS num_accounts
FROM bank_customers c JOIN accounts a ON c.cust_id=a.cust_id
GROUP BY c.cust_id, c.name HAVING COUNT(*) > 1;
```

C4

```
SELECT acc_id, balance FROM accounts
WHERE balance = (SELECT MAX(balance) FROM accounts);
```

C5

```
SELECT acc_id, txn_date, amount,
       SUM(amount) OVER (PARTITION BY acc_id ORDER BY txn_date) AS running_total
FROM transactions;
```

Section D

D

```
WITH net AS (
  SELECT acc_id,
         SUM(CASE WHEN txn_type='Credit' THEN amount ELSE -amount END) AS net_move
  FROM transactions GROUP BY acc_id
)
SELECT acc_id, net_move FROM net
WHERE net_move > 0 ORDER BY net_move DESC;
```

Solutions — Practice Paper 5

Hospital & Patient Records

Model answers below. Several questions have more than one correct approach; these are reference solutions in MySQL 8.0+ syntax.

Section A

A1

```
ALTER TABLE patients ADD COLUMN phone VARCHAR(15);
```

A2

```
INSERT INTO doctors (name,specialty,fee) VALUES ('Dr. Sharma','Neurology',1000);
```

A3

```
UPDATE appointments SET status='Completed' WHERE appt_id=7;
```

A4

```
DELETE FROM appointments WHERE status='Cancelled';
```

A5

```
-- appointments.patient_id -> patients.patient_id and  
-- appointments.doctor_id   -> doctors.doctor_id (two FKs).
```

Section B

B1

```
SELECT * FROM patients WHERE age > 40 ORDER BY age DESC;
```

B2

```
SELECT * FROM doctors WHERE specialty='Cardiology' AND fee > 800;
```

B3

```
SELECT name, fee, ROUND(fee*1.18,2) AS total_fee FROM doctors;
```

B4

```
SELECT * FROM patients WHERE gender='F' AND city IN ('Mumbai','Delhi');
```

B5

```
SELECT name, age,
```

```
CASE WHEN age<18 THEN 'Child'
      WHEN age<60 THEN 'Adult' ELSE 'Senior' END AS category
FROM patients;
```

Section C

C1

```
SELECT d.name, COUNT(a.appt_id) AS appts
FROM doctors d LEFT JOIN appointments a ON d.doctor_id=a.doctor_id
GROUP BY d.doctor_id, d.name;
```

C2

```
SELECT d.specialty, SUM(d.fee) AS revenue
FROM appointments a JOIN doctors d ON a.doctor_id=d.doctor_id
WHERE a.status='Completed'
GROUP BY d.specialty;
```

C3

```
SELECT d.name, COUNT(*) AS completed
FROM doctors d JOIN appointments a ON d.doctor_id=a.doctor_id
WHERE a.status='Completed'
GROUP BY d.name HAVING COUNT(*) > 1;
```

C4

```
SELECT p.name, COUNT(a.appt_id) AS appts
FROM patients p LEFT JOIN appointments a ON p.patient_id=a.patient_id
GROUP BY p.patient_id, p.name;
```

C5

```
SELECT name, fee FROM doctors
WHERE fee > (SELECT AVG(fee) FROM doctors);
```

Section D

D

```
WITH r AS (
  SELECT d.name, d.specialty, d.fee,
         RANK() OVER (PARTITION BY d.specialty ORDER BY d.fee DESC) AS rnk,
         (SELECT COUNT(*) FROM appointments a WHERE a.doctor_id=d.doctor_id) AS appts
  FROM doctors d
)
SELECT name, specialty, fee, appts FROM r WHERE rnk=1;
```

Solutions — Practice Paper 6

School: Students, Courses & Grades

Model answers below. Several questions have more than one correct approach; these are reference solutions in MySQL 8.0+ syntax.

Section A

A1

```
ALTER TABLE students ADD COLUMN email VARCHAR(60);
```

A2

```
INSERT INTO courses (course_name,credits,teacher)
VALUES ('Chemistry',4,'Dr. Nair');
```

A3

```
UPDATE enrollments SET marks = marks+5 WHERE student_id=4 AND course_id=1;
```

A4

```
-- (student_id,course_id) is composite because a student can take
-- many courses and a course has many students; the pair is unique.
```

A5

```
DELETE FROM enrollments WHERE marks < 40;
```

Section B

B1

```
SELECT name FROM students WHERE grade_level=12 ORDER BY name;
```

B2

```
SELECT * FROM enrollments WHERE marks >= 80;
```

B3

```
SELECT course_name, credits*10 AS weighted FROM courses;
```

B4

```
SELECT name FROM students WHERE city IN ('Mumbai','Pune');
```

B5

```
SELECT student_id, course_id, marks,  
       CASE WHEN marks >= 85 THEN 'A'  
            WHEN marks >= 70 THEN 'B' ELSE 'C' END AS grade  
FROM enrollments;
```

Section C

C1

```
SELECT c.course_name, ROUND(AVG(e.marks),1) AS avg_marks  
FROM courses c JOIN enrollments e ON c.course_id=e.course_id  
GROUP BY c.course_name;
```

C2

```
SELECT s.name, SUM(e.marks) AS total_marks  
FROM students s JOIN enrollments e ON s.student_id=e.student_id  
GROUP BY s.student_id, s.name;
```

C3

```
SELECT c.course_name, AVG(e.marks) AS avg_marks  
FROM courses c JOIN enrollments e ON c.course_id=e.course_id  
GROUP BY c.course_name HAVING AVG(e.marks) > 80;
```

C4

```
SELECT s.name, COUNT(e.course_id) AS num_courses  
FROM students s LEFT JOIN enrollments e ON s.student_id=e.student_id  
GROUP BY s.student_id, s.name;
```

C5

```
SELECT DISTINCT s.name FROM students s JOIN enrollments e  
ON s.student_id=e.student_id  
WHERE e.marks > (SELECT AVG(marks) FROM enrollments);
```

Section D

D

```
WITH r AS (  
  SELECT s.name AS student, c.course_name, e.marks,  
         RANK() OVER (PARTITION BY e.course_id ORDER BY e.marks DESC) AS rnk  
  FROM enrollments e  
  JOIN students s ON e.student_id=s.student_id  
  JOIN courses c ON e.course_id=c.course_id  
)  
SELECT student, course_name, marks FROM r WHERE rnk=1;
```

Solutions — Practice Paper 7

Food Delivery Analytics

Model answers below. Several questions have more than one correct approach; these are reference solutions in MySQL 8.0+ syntax.

Section A

A1

```
ALTER TABLE restaurants ADD COLUMN veg_only VARCHAR(3);
```

A2

```
INSERT INTO restaurants (name,cuisine,city,rating)
VALUES ('Taco Town','Mexican','Pune',4.1);
```

A3

```
UPDATE restaurants SET rating=4.1 WHERE name='Wok Star';
```

A4

```
DELETE FROM food_orders WHERE amount < 250;
```

A5

```
-- deliveries.order_id is a FK to food_orders.order_id
-- (one delivery per order).
```

Section B

B1

```
SELECT * FROM restaurants WHERE cuisine='Indian' AND rating >= 4.5;
```

B2

```
SELECT * FROM food_orders
WHERE order_date BETWEEN '2024-02-01' AND '2024-02-29'
ORDER BY amount DESC;
```

B3

```
SELECT order_id, amount, ROUND(amount*1.05,2) AS total FROM food_orders;
```

B4

```
SELECT * FROM restaurants WHERE city IN ('Mumbai','Delhi');
```

B5


```
SELECT order_id, amount,  
       CASE WHEN amount >= 500 THEN 'Large' ELSE 'Regular' END AS bucket  
FROM food_orders;
```

Section C

C1

```
SELECT r.name, COUNT(o.order_id) AS orders, SUM(o.amount) AS revenue  
FROM restaurants r LEFT JOIN food_orders o ON r.rest_id=o.rest_id  
GROUP BY r.rest_id, r.name;
```

C2

```
SELECT d.agent, ROUND(AVG(d.minutes),1) AS avg_minutes  
FROM deliveries d GROUP BY d.agent;
```

C3

```
SELECT r.cuisine, SUM(o.amount) AS total  
FROM restaurants r JOIN food_orders o ON r.rest_id=o.rest_id  
GROUP BY r.cuisine HAVING SUM(o.amount) > 1000;
```

C4

```
SELECT r.name, COUNT(o.order_id) AS orders  
FROM restaurants r LEFT JOIN food_orders o ON r.rest_id=o.rest_id  
GROUP BY r.rest_id, r.name;
```

C5

```
SELECT name, rating FROM restaurants  
WHERE rating > (SELECT AVG(rating) FROM restaurants);
```

Section D

D

```
WITH rev AS (  
  SELECT r.name, r.city, SUM(o.amount) AS total_revenue,  
         RANK() OVER (PARTITION BY r.city ORDER BY SUM(o.amount) DESC) AS rnk  
  FROM restaurants r JOIN food_orders o ON r.rest_id=o.rest_id  
  GROUP BY r.rest_id, r.name, r.city  
)  
SELECT name, city, total_revenue FROM rev WHERE rnk=1;
```

Solutions — Practice Paper 8

Video Streaming Platform

Model answers below. Several questions have more than one correct approach; these are reference solutions in MySQL 8.0+ syntax.

Section A

A1

```
ALTER TABLE users ADD COLUMN last_login DATE;
```

A2

```
INSERT INTO titles (name,genre,release_year,duration_min)
VALUES ('Deep Blue','Documentary',2023,88);
```

A3

```
UPDATE users SET plan='Standard' WHERE user_id=2;
```

A4

```
DELETE FROM titles WHERE release_year < 2021;
```

A5

```
-- views.user_id -> users.user_id and views.title_id -> titles.title_id.
```

Section B

B1

```
SELECT * FROM titles WHERE release_year=2023 AND duration_min>100
ORDER BY duration_min;
```

B2

```
SELECT * FROM users WHERE plan='Premium' AND country IN ('India','UK');
```

B3

```
SELECT name, ROUND(duration_min/60,2) AS hours FROM titles;
```

B4

```
SELECT DISTINCT genre FROM titles;
```

B5

```
SELECT name, duration_min,  
       CASE WHEN duration_min>=120 THEN 'Long' ELSE 'Short' END AS length  
FROM titles;
```

Section C

C1

```
SELECT u.name, SUM(v.minutes_watched) AS total_minutes  
FROM users u JOIN views v ON u.user_id=v.user_id  
GROUP BY u.user_id, u.name;
```

C2

```
SELECT t.genre, COUNT(*) AS views, SUM(v.minutes_watched) AS minutes  
FROM titles t JOIN views v ON t.title_id=v.title_id  
GROUP BY t.genre;
```

C3

```
SELECT t.genre, SUM(v.minutes_watched) AS minutes  
FROM titles t JOIN views v ON t.title_id=v.title_id  
GROUP BY t.genre HAVING SUM(v.minutes_watched) > 150;
```

C4

```
SELECT t.name, COUNT(v.view_id) AS times_watched  
FROM titles t LEFT JOIN views v ON t.title_id=v.title_id  
GROUP BY t.title_id, t.name;
```

C5

```
SELECT name, duration_min FROM titles  
WHERE duration_min > (SELECT AVG(duration_min) FROM titles);
```

Section D

D

```
WITH w AS (  
  SELECT u.name AS user, t.name AS title, v.minutes_watched,  
         ROW_NUMBER() OVER (PARTITION BY u.user_id  
                             ORDER BY v.minutes_watched DESC) AS rn  
  FROM views v  
  JOIN users u ON v.user_id=u.user_id  
  JOIN titles t ON v.title_id=t.title_id  
)  
SELECT user, title, minutes_watched FROM w WHERE rn=1;
```

Solutions — Practice Paper 9

Airline Booking System

Model answers below. Several questions have more than one correct approach; these are reference solutions in MySQL 8.0+ syntax.

Section A

A1

```
ALTER TABLE passengers ADD COLUMN email VARCHAR(60);
```

A2

```
INSERT INTO flights (flight_no,origin,destination,fare,seats)
VALUES ('AI606','Bengaluru','Mumbai',5000,160);
```

A3

```
UPDATE flights SET fare = fare*1.10 WHERE origin='Mumbai';
```

A4

```
DELETE FROM bookings WHERE booking_date < '2024-02-01';
```

A5

```
-- bookings.passenger_id -> passengers.passenger_id and
-- bookings.flight_id -> flights.flight_id (two FKs).
```

Section B

B1

```
SELECT * FROM flights WHERE fare > 5000 ORDER BY fare DESC;
```

B2

```
SELECT * FROM passengers
WHERE tier IN ('Gold','Platinum') AND city IN ('Mumbai','Delhi');
```

B3

```
SELECT flight_no, CONCAT(origin,' -> ',destination) AS route FROM flights;
```

B4

```
SELECT DISTINCT origin FROM flights;
```

B5

```
SELECT flight_no, fare,  
       CASE WHEN fare>=6000 THEN 'Premium' ELSE 'Economy' END AS class  
FROM flights;
```

Section C

C1

```
SELECT f.flight_no, COUNT(b.booking_id) AS bookings, SUM(b.amount) AS revenue  
FROM flights f LEFT JOIN bookings b ON f.flight_id=b.flight_id  
GROUP BY f.flight_id, f.flight_no;
```

C2

```
SELECT p.tier, SUM(b.amount) AS spent  
FROM passengers p JOIN bookings b ON p.passenger_id=b.passenger_id  
GROUP BY p.tier;
```

C3

```
SELECT f.flight_no, COUNT(*) AS bookings  
FROM flights f JOIN bookings b ON f.flight_id=b.flight_id  
GROUP BY f.flight_no HAVING COUNT(*) > 1;
```

C4

```
SELECT p.name, COUNT(b.booking_id) AS bookings  
FROM passengers p LEFT JOIN bookings b ON p.passenger_id=b.passenger_id  
GROUP BY p.passenger_id, p.name;
```

C5

```
SELECT flight_no, fare FROM flights  
WHERE fare > (SELECT AVG(fare) FROM flights);
```

Section D

D

```
SELECT f.flight_no, SUM(b.amount) AS revenue,  
       RANK() OVER (ORDER BY SUM(b.amount) DESC) AS rev_rank,  
       ROUND(100*SUM(b.amount)/SUM(SUM(b.amount)) OVER ()),1) AS pct_of_total  
FROM flights f JOIN bookings b ON f.flight_id=b.flight_id  
GROUP BY f.flight_id, f.flight_no;
```

Solutions — Practice Paper 10

Warehouse & Inventory Management

Model answers below. Several questions have more than one correct approach; these are reference solutions in MySQL 8.0+ syntax.

Section A

A1

```
ALTER TABLE inventory ADD CONSTRAINT chk_qty CHECK (quantity >= 0);
```

A2

```
INSERT INTO inventory (item_name,supplier_id,unit_cost,quantity)
VALUES ('Monitor',1,12000,30);
```

A3

```
UPDATE inventory SET quantity = quantity-50 WHERE item_id=2;
```

A4

```
DELETE FROM inventory WHERE quantity=0;
```

A5

```
-- With ON DELETE RESTRICT, MySQL blocks deleting a supplier that
-- still has inventory rows referencing it (referential integrity).
```

Section B

B1

```
SELECT * FROM inventory WHERE unit_cost > 1000 ORDER BY unit_cost DESC;
```

B2

```
SELECT i.* FROM inventory i JOIN suppliers s ON i.supplier_id=s.supplier_id
WHERE s.city IN ('Mumbai','Pune');
```

B3

```
SELECT item_name, unit_cost*quantity AS stock_value FROM inventory;
```

B4

```
SELECT item_name, quantity FROM inventory WHERE quantity < 50;
```

B5

```
SELECT item_name, quantity,
       CASE WHEN quantity=0 THEN 'Out'
            WHEN quantity<50 THEN 'Low' ELSE 'OK' END AS stock_status
FROM inventory;
```

Section C

C1

```
SELECT s.name, SUM(i.unit_cost*i.quantity) AS stock_value
FROM suppliers s JOIN inventory i ON s.supplier_id=i.supplier_id
GROUP BY s.supplier_id, s.name;
```

C2

```
SELECT i.item_name, COUNT(m.move_id) AS moves
FROM inventory i LEFT JOIN stock_moves m ON i.item_id=m.item_id
GROUP BY i.item_id, i.item_name;
```

C3

```
SELECT s.name, AVG(i.unit_cost) AS avg_cost
FROM suppliers s JOIN inventory i ON s.supplier_id=i.supplier_id
GROUP BY s.name HAVING AVG(i.unit_cost) > 1000;
```

C4

```
SELECT i.item_name,
       SUM(CASE WHEN m.move_type='IN' THEN m.qty ELSE 0 END) AS total_in,
       SUM(CASE WHEN m.move_type='OUT' THEN m.qty ELSE 0 END) AS total_out
FROM inventory i LEFT JOIN stock_moves m ON i.item_id=m.item_id
GROUP BY i.item_id, i.item_name;
```

C5

```
SELECT item_name, unit_cost FROM inventory
WHERE unit_cost > (SELECT AVG(unit_cost) FROM inventory);
```

Section D

D

```
WITH net AS (
  SELECT i.item_name,
         SUM(CASE WHEN m.move_type='IN' THEN m.qty ELSE -m.qty END) AS net_change
  FROM inventory i JOIN stock_moves m ON i.item_id=m.item_id
  GROUP BY i.item_id, i.item_name
)
SELECT item_name, net_change FROM net
WHERE net_change < 0 ORDER BY net_change ASC;
```

APPENDIX C

Answers to Chapter Practice Questions

Reference answers for the hands-on querying chapters. They assume the Setup Scripts shown in each chapter have been run.

Chapters 7–11 — DML & Core SELECT

Chapter 7 — INSERT, UPDATE, DELETE

Q1

```
INSERT INTO products (name,price) VALUES  
('Ruler',12),('Glue',35),('Scissors',80);
```

Q2

```
UPDATE products SET stock = stock+100 WHERE price < 50;
```

Q3

```
UPDATE products SET name='Spiral Notebook', price=60 WHERE product_id=2;
```

Q4

```
DELETE FROM products WHERE price > 100;
```

Q5

```
INSERT INTO bestsellers (name,price)  
SELECT name, price FROM products WHERE stock > 200;
```

Chapter 8 — SELECT & Clauses

Q1

```
SELECT name, city FROM employees;
```

Q2

```
SELECT name, salary/12 AS monthly_take_home FROM employees;
```

Q3

```
SELECT name, salary FROM employees WHERE salary > 50000;
```

Q4


```
SELECT name, hire_date FROM employees WHERE hire_date >= '2022-01-01';
```

Q5

```
-- Alias 'annual' is not visible in WHERE (SELECT runs after WHERE).  
SELECT name, salary*12 AS annual FROM employees WHERE salary*12 > 600000;
```

Chapter 9 — Operators

Q1

```
SELECT name, salary FROM employees WHERE salary BETWEEN 45000 AND 60000;
```

Q2

```
SELECT name FROM employees WHERE city IN ('Mumbai','Delhi','Pune');
```

Q3

```
SELECT name FROM employees WHERE name LIKE '%a';
```

Q4

```
SELECT name FROM employees WHERE name LIKE '%r%';
```

Q5

```
SELECT name FROM employees WHERE commission IS NULL AND dept_id = 30;
```

Chapter 10 — NULL Handling

Q1

```
SELECT name, IFNULL(commission,0) AS commission FROM employees;
```

Q2

```
SELECT name, salary + IFNULL(commission,0) AS total_pay FROM employees;
```

Q3

```
SELECT COUNT(*) AS null_comm FROM employees WHERE commission IS NULL;
```

Q4

```
-- '= NULL' is never TRUE because NULL means 'unknown';  
-- must use IS NULL.
```

Q5

```
SELECT name, salary / NULLIF(commission,0) AS ratio FROM employees;
```

Chapter 11 — Sorting, DISTINCT, LIMIT

Q1

```
SELECT name, salary FROM employees ORDER BY salary DESC;
```

Q2

```
SELECT DISTINCT city FROM employees;
```

Q3

```
SELECT name, salary FROM employees ORDER BY salary DESC LIMIT 3;
```

Q4

```
SELECT name, city, salary FROM employees ORDER BY city ASC, salary DESC;
```

Q5

```
SELECT name FROM employees ORDER BY name LIMIT 2 OFFSET 3;
```

Chapters 12–15 — Summarising Data

Chapter 12 — Aggregates & GROUP BY

Q1

```
SELECT COUNT(*) AS total_sales, SUM(amount) AS revenue FROM sales;
```

Q2

```
SELECT product, SUM(qty) AS units FROM sales GROUP BY product;
```

Q3

```
SELECT region, ROUND(AVG(amount),0) AS avg_amt FROM sales GROUP BY region;
```

Q4

```
SELECT region, MAX(amount) AS biggest FROM sales GROUP BY region;
```

Q5

```
SELECT region, product, SUM(qty) AS units  
FROM sales GROUP BY region, product;
```

Chapter 13 — HAVING

Q1

```
SELECT product, SUM(amount) AS rev FROM sales
GROUP BY product HAVING SUM(amount) > 5000;
```

Q2

```
SELECT region, COUNT(*) AS n FROM sales
GROUP BY region HAVING COUNT(*) > 2;
```

Q3

```
SELECT region, SUM(qty) AS q FROM sales
WHERE product='Mouse' GROUP BY region HAVING SUM(qty) > 4;
```

Q4

```
-- WHERE filters rows before grouping; HAVING filters groups after.
```

Q5

```
SELECT product, AVG(amount) AS a FROM sales
GROUP BY product HAVING AVG(amount) > 10000 ORDER BY a DESC;
```

Chapter 14 — Built-in Functions

Q1

```
SELECT UPPER(TRIM(full_name)) AS name FROM people;
```

Q2

```
SELECT SUBSTRING_INDEX(email, '@', -1) AS domain FROM people;
```

Q3

```
SELECT ROUND(salary, 0) AS rupees, ROUND(salary, -2) AS nearest_100 FROM people;
```

Q4

```
SELECT full_name, DATEDIFF(CURDATE(), joined_on) AS days FROM people;
```

Q5

```
SELECT DATE_FORMAT(joined_on, '%d %M %Y') AS joined FROM people;
```

Chapter 15 — CASE

Q1

```
SELECT name, CASE WHEN hire_date < '2022-01-01' THEN 'Senior'
                ELSE 'Recent' END AS seniority FROM employees;
```

Q2

```
SELECT name, CASE WHEN salary >= 70000 THEN 'High'
                WHEN salary >= 50000 THEN 'Medium' ELSE 'Entry' END AS band
FROM employees;
```

Q3

```
SELECT name, CASE dept_id WHEN 10 THEN 'Sales' WHEN 20 THEN 'Engineering'
                WHEN 30 THEN 'Support' WHEN 40 THEN 'Marketing' ELSE 'None' END AS dept
FROM employees;
```

Q4

```
SELECT SUM(CASE WHEN salary >= 70000 THEN 1 ELSE 0 END) AS high,
       SUM(CASE WHEN salary BETWEEN 50000 AND 69999 THEN 1 ELSE 0 END) AS medium,
       SUM(CASE WHEN salary < 50000 THEN 1 ELSE 0 END) AS entry FROM employees;
```

Q5

```
SELECT name, city FROM employees
ORDER BY CASE WHEN city='Mumbai' THEN 0 ELSE 1 END, name;
```

Chapters 16–20 — Combining & Advanced

Chapter 16 — Joins

Q1

```
SELECT c.name, o.amount FROM customers c
JOIN orders o ON c.customer_id=o.customer_id;
```

Q2

```
SELECT c.name, COUNT(o.order_id) AS orders FROM customers c
LEFT JOIN orders o ON c.customer_id=o.customer_id
GROUP BY c.customer_id, c.name;
```

Q3

```
SELECT c.name FROM customers c
LEFT JOIN orders o ON c.customer_id=o.customer_id WHERE o.order_id IS NULL;
```

Q4

```
SELECT c.city, SUM(o.amount) AS revenue FROM customers c
JOIN orders o ON c.customer_id=o.customer_id GROUP BY c.city;
```

Q5

```
SELECT c.name, o.amount FROM orders o
JOIN customers c ON o.customer_id=c.customer_id
ORDER BY o.amount DESC LIMIT 1;
```

Chapter 17 — Set Operations

Q1

```
SELECT name FROM online_customers UNION SELECT name FROM store_customers;
```

Q2

```
SELECT name FROM online_customers UNION ALL SELECT name FROM store_customers;
-- UNION ALL when duplicates matter or for speed (no de-dup).
```

Q3

```
SELECT name FROM online_customers INTERSECT SELECT name FROM store_customers;
```

Q4

```
SELECT name FROM online_customers EXCEPT SELECT name FROM store_customers;
```

Q5

```
SELECT DISTINCT o.name FROM online_customers o
JOIN store_customers s ON o.name=s.name;
```

Chapter 18 — Subqueries

Q1

```
SELECT name, salary FROM staff
WHERE salary > (SELECT AVG(salary) FROM staff);
```

Q2

```
SELECT name, salary FROM staff
WHERE salary = (SELECT MAX(salary) FROM staff);
```

Q3

```
SELECT name FROM staff WHERE dept IN
(SELECT dept FROM staff WHERE name='Kabir');
```

Q4

```
SELECT s1.name, s1.dept FROM staff s1 WHERE s1.salary >
(SELECT AVG(s2.salary) FROM staff s2 WHERE s2.dept=s1.dept);
```

Q5

```
SELECT dept, avg_sal FROM
  (SELECT dept, AVG(salary) avg_sal FROM staff GROUP BY dept) d
WHERE avg_sal > 50000;
```

Chapter 19 — CTEs

Q1

```
WITH da AS (SELECT dept, AVG(salary) a FROM staff GROUP BY dept)
SELECT s.name FROM staff s JOIN da ON s.dept=da.dept WHERE s.salary>da.a;
```

Q2

```
WITH tech AS (SELECT * FROM staff WHERE dept='Tech'),
    hi AS (SELECT * FROM tech WHERE salary>60000)
SELECT name, salary FROM hi;
```

Q3

```
WITH RECURSIVE n AS (SELECT 1 x UNION ALL SELECT x+1 FROM n WHERE x<20)
SELECT x FROM n;
```

Q4

```
WITH RECURSIVE m AS (SELECT 5 v UNION ALL SELECT v+5 FROM m WHERE v<40)
SELECT v FROM m;
```

Q5

```
-- A CTE can be referenced multiple times and can be recursive;
-- a derived table cannot.
```

Chapter 20 — Window Functions

Q1

```
SELECT name, salary, RANK() OVER (ORDER BY salary DESC) AS rnk FROM staff;
```

Q2

```
SELECT name, dept, ROW_NUMBER() OVER
  (PARTITION BY dept ORDER BY salary DESC) AS rn FROM staff;
```

Q3

```
SELECT name, ROUND(100*salary/SUM(salary) OVER (PARTITION BY dept),1)
  AS pct_of_dept FROM staff;
```

Q4

```
SELECT name, salary, SUM(salary) OVER (ORDER BY id) AS running FROM staff;
```

Q5

```
WITH r AS (SELECT name,dept,salary,  
    ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) rn FROM staff)  
SELECT name,dept,salary FROM r WHERE rn=1;
```

APPENDIX B

MySQL Quick Reference

Command & Function Cheat Sheet

DDL — Structure

```
CREATE DATABASE db;          USE db;
CREATE TABLE t (col TYPE constraints, ...);
ALTER TABLE t ADD COLUMN c TYPE;      ALTER TABLE t DROP COLUMN c;
ALTER TABLE t MODIFY c NEWTYPE;      RENAME TABLE t TO t2;
TRUNCATE TABLE t;  DROP TABLE t;    DROP DATABASE db;
```

DML — Data

```
INSERT INTO t (c1,c2) VALUES (v1,v2), (v3,v4);
UPDATE t SET c1 = v WHERE cond;
DELETE FROM t WHERE cond;
INSERT INTO t2 (c) SELECT c FROM t WHERE cond;
```

SELECT skeleton (logical order)

```
SELECT  col, AGG(col)
FROM    t1 JOIN t2 ON t1.k = t2.k
WHERE   row_condition
GROUP BY col
HAVING  group_condition
ORDER BY col [ASC|DESC]
LIMIT  n OFFSET m;
```

Operators

Result:

Operator	Meaning
= < > < >= <=	Comparison
AND OR NOT	Logical
BETWEEN a AND b	Inclusive range
IN (list)	Membership
LIKE '%_'	Pattern (% any, _ one)
IS NULL / IS NOT NULL	NULL test

Aggregate & NULL

```
COUNT(*)  COUNT(col)  SUM()  AVG()  MIN()  MAX()
IFNULL(x,0)  COALESCE(a,b,c)  NULLIF(a,b)
```


String functions

```
UPPER LOWER LENGTH CONCAT(a,b) SUBSTRING(s,pos,len)
TRIM LTRIM RTRIM REPLACE(s,a,b) LEFT(s,n) RIGHT(s,n)
SUBSTRING_INDEX(s,delim,count) LOCATE(sub,s)
```

Numeric & Date functions

```
ROUND(x,d) CEIL(x) FLOOR(x) ABS(x) MOD(a,b) POWER(a,b)
NOW() CURDATE() YEAR(d) MONTH(d) DAY(d)
DATEDIFF(a,b) DATE_ADD(d,INTERVAL n UNIT) DATE_FORMAT(d,'%d-%b-%Y')
TIMESTAMPDIFF(YEAR,a,b)
```

Joins

```
A INNER JOIN B ON A.k=B.k    -- matches only
A LEFT JOIN B ON A.k=B.k     -- all A + matches
A RIGHT JOIN B ON A.k=B.k    -- all B + matches
A CROSS JOIN B               -- every combination
-- anti-join: LEFT JOIN ... WHERE B.k IS NULL
```

Set operations

```
q1 UNION q2      -- distinct rows of both
q1 UNION ALL q2  -- keep duplicates
q1 INTERSECT q2  -- rows in both (8.0.31+)
q1 EXCEPT q2   -- in q1 not in q2 (8.0.31+)
```

Subqueries, CTE, Windows

```
-- scalar / IN / correlated / EXISTS subqueries
WHERE col > (SELECT AVG(col) FROM t)

WITH cte AS (SELECT ...) SELECT ... FROM cte;
WITH RECURSIVE r AS (anchor UNION ALL recursive) SELECT * FROM r;

ROW_NUMBER() RANK() DENSE_RANK() OVER (PARTITION BY p ORDER BY o)
SUM(x) OVER (PARTITION BY p ORDER BY o)  -- running total
LAG(x) LEAD(x) OVER (ORDER BY o)
```

Views, Procedures, Indexes

```
CREATE OR REPLACE VIEW v AS SELECT ...;  DROP VIEW v;

DELIMITER //
```

```
CREATE PROCEDURE p(IN x INT) BEGIN ... END //  
DELIMITER ; CALL p(1);  
  
CREATE INDEX idx ON t(col); EXPLAIN SELECT ...;
```

Constraints

Result:

Constraint	Meaning
NOT NULL	value required
UNIQUE	no duplicates
PRIMARY KEY	unique + not null
FOREIGN KEY	references another table
CHECK (cond)	value must satisfy cond
DEFAULT val	used when none supplied

End of Notes

*Keep practising — run every query, break things, and fix them.
That is how SQL fluency is built.*